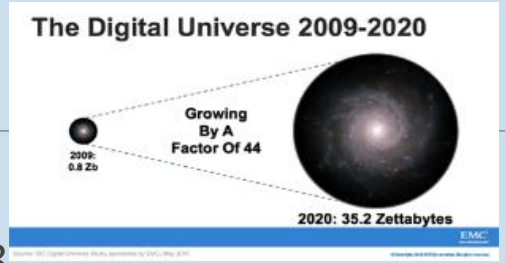
Big Data Analytics
&
AI – An Overview

Big Data

Data that cannot be organized, analysed and visualized by the traditional methods

Characteristics of Big Data

1. Volume



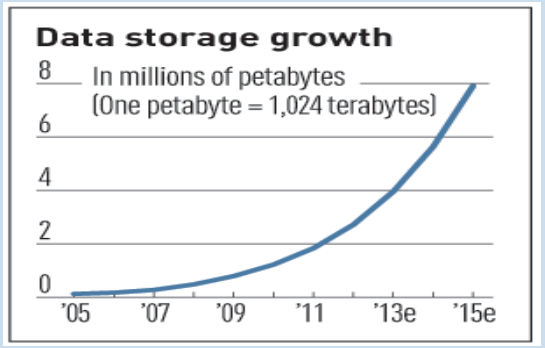
Data volume is increasing exponentially.

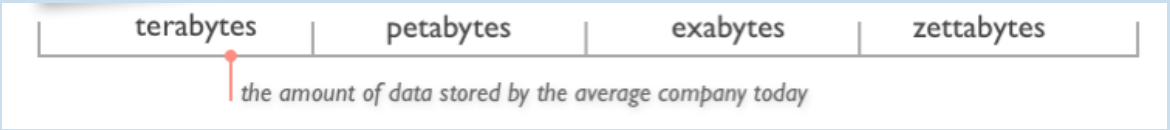
In the next five years, these files will grow by a factor of 8.

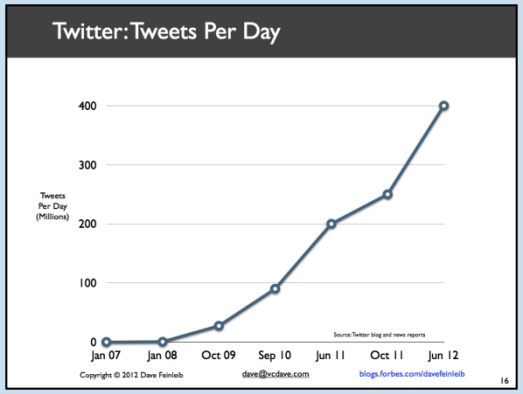
Ebay 90PB, Facebook 50 billion photos

Large Hadron collider 40TB/sec

The amount of information individuals create themselves in form of writing documents, taking pictures, downloading music, etc, **is far less than the amount of information being created about them.**







Characteristics of Big Data

2. Velocity

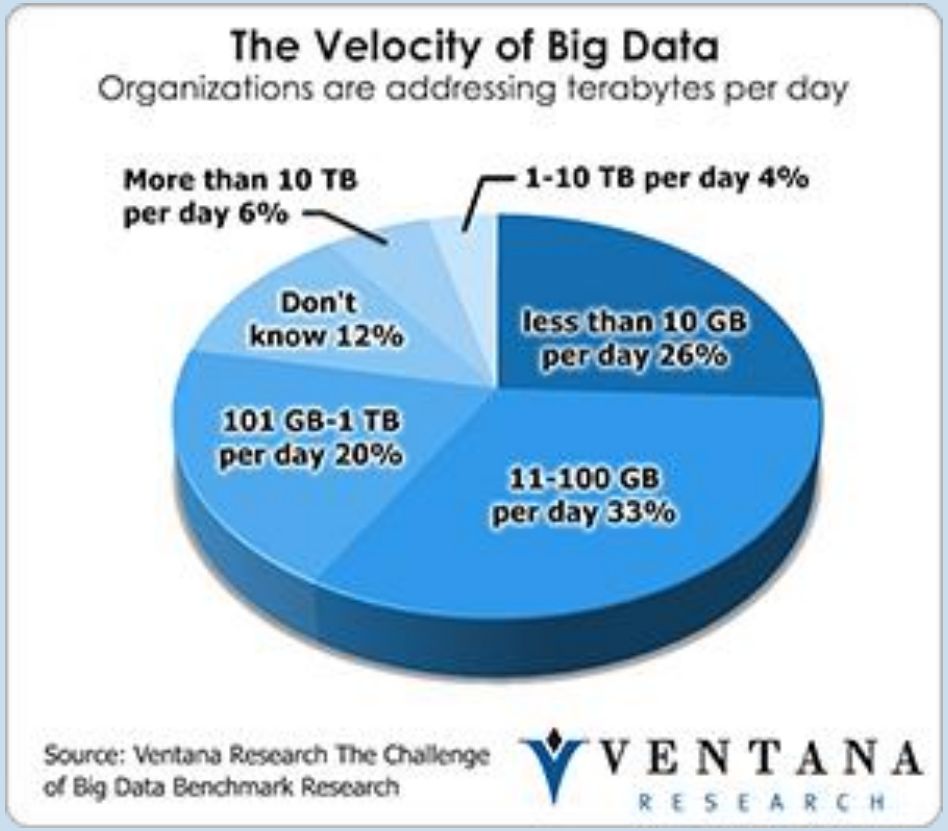


How quickly data is being **produced** and how quickly the data must be **processed** to meet the demand for extracting useful information.

Sources of high-velocity data such as log files of websites, devices that log events, etc.

Social media: Twitter **500mil T/day** (peak150,000T/s), Facebook 800mil active users, **4,5 bilion likes/day**. The value of this data degrades over time.

Useful information must be extracted in a timely manner otherwise it looses meaning.



Characteristics of Big Data

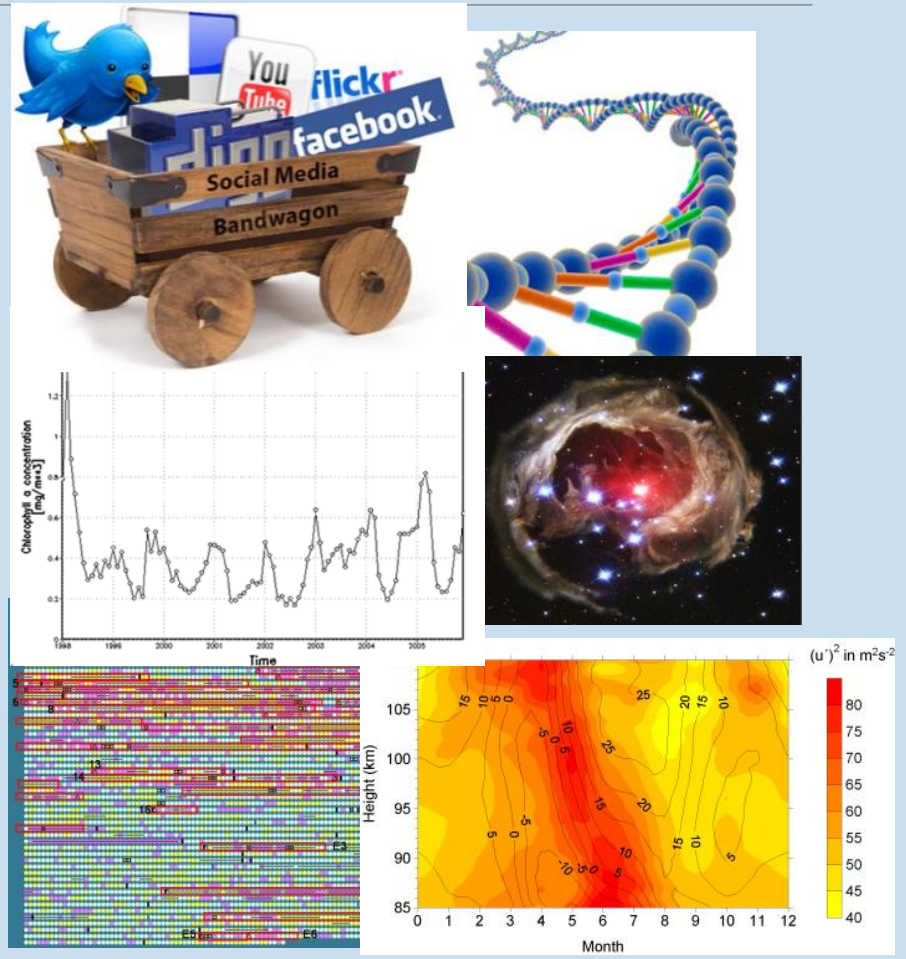
3. Variety

A single application can generate and collect many types of data.

many format and types: structured, unstructured, semi-structured, text (even in different languages), media, etc.

problems not only for storing and efficient management but also for mining and for analyzing data.

To extract the knowledge all these types of data need to be linked together.



Characteristics of Big Data

4. Veracity

Relates to **reliability** of data and predictability of inherently imprecise data with **noise and abnormality**.

data must be **meaningful** to the problem being analyzed.

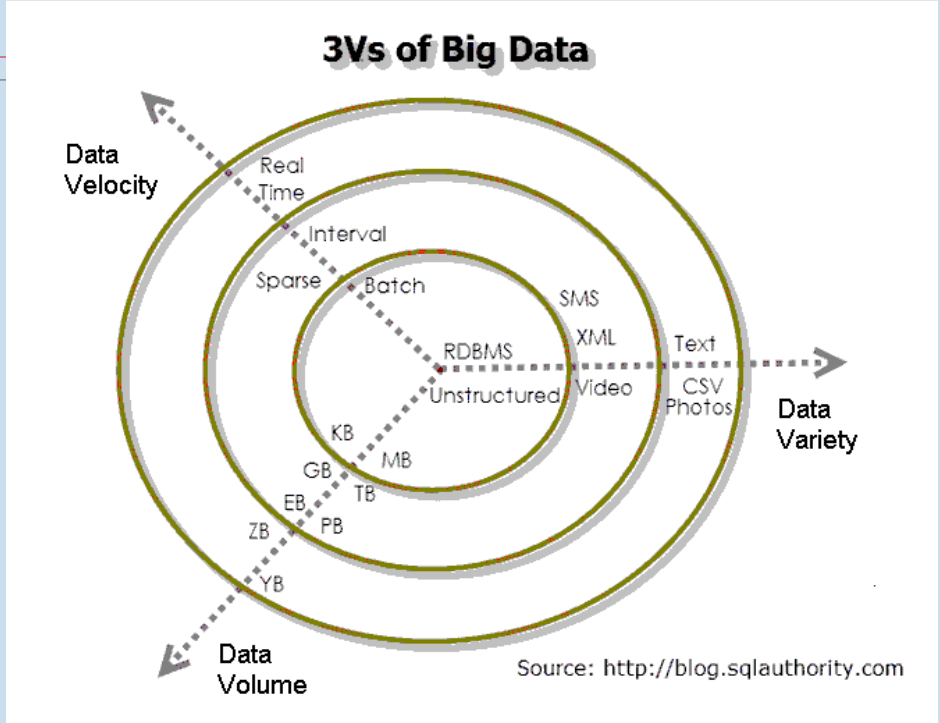
the **biggest challenge is** to keep your data clean .



Characteristics of Big Data

Variety and Velocity are working against the Veracity of the data. They decrease the ability to cleanse the data before the information is extracted.

5th V: Value - is data worthwhile and has value for business.



- **6th V: Variability** - different meanings of data
- **7th V: Volatility -** how long data is valid and are relevant to the analysis and decisions.

What Industries Can Do with Big Data



Financial Services
- Fraud detection
- 360° View of the Customer

Utilities
- Weather analysis
- Smart grid management

Transportation
- Logistics optimization
- Traffic congestion

IT
- System Log Analysis
- Cybersecurity
- Outage prevention
- Resource Prediction
- Warehouse Integration

Health & Life Sciences
- Epidemic early warning
- ICU monitoring

Retail
- Marketing Campaign Efficiency
- Targeted Marketing
- MicroSegmentation

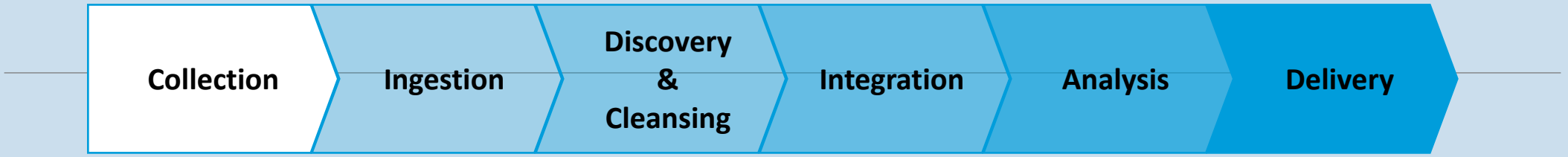
Telecommunications
- Geomapping / marketing
- Network monitoring

Law Enforcement
- Multimodal surveillance
- Cyber security detection

Multiple Industries
- Customer Retention
- Customer Acquisition

Manufacturing
- Manufacturing Efficiency

Big Data Value Chain



Collection – Structured, unstructured and semi-structured data from multiple sources

Ingestion – loading vast amounts of data onto a single data store

Discovery & Cleansing – understanding format and content; clean up and formatting

Integration – linking, entity extraction, entity resolution, indexing and data fusion

Analysis – Intelligence, statistics, predictive and text analytics, machine learning

Delivery – querying, visualization, real time delivery on enterprise-class availability

Need for Standardized Approaches At Each Step

Collection

What is collecting all this data?

WEB BROWSERS

Microsoft's Internet Explorer



Mozilla's FireFox
(Non-profit foundation, used to be Netscape)



Google's Chrome



Apple's Safari



Time-Warner's AOL Explorer



SEARCH ENGINES

Google's



Microsoft's



Yahoo's



IAC Search's



What is collecting all this data?

SMARTPHONES & APPS

Apple's iPhone
(Apple O/S)



Samsung, HTC.
Nokia, Motorola
(Android O/S)



RIM Corp's Blackberry
(BlackBerry O/S)



TABLET COMPUTERS & APPS

Apple's iPad



Samsung's Galaxy



Amazon's Kindle Fire



What is collecting all this data?

GAMES BOXES AND GPS SYSTEMS









INTERNET SERVICE PROVIDERS









What is collecting all this data?

MOVIE RENTAL SITES

HDTV'S AND BLU-RAY PLAYERS WITH BUILT-IN INTERNET CONNECTIVITY





NETFLIX

redbox

amazon instant video

Connecting

What is collecting all this data?

HOSPITALS & OTHER MEDICAL SYSTEMS

Pharmacies
Laboratories
Imaging Centers
Emergency Medical Services (EMS)
Hospital Information Systems
Doc-in-a-Box
Electronic Medical Records
Blood Banks
Birth & Death Records

BANKING & PHONE SYSTEMS



verizon

Can you hear me now?
(Heh heh heh!)

Sprint

T··Mobi

at&t

What is collecting all this data?

A REAL PAIN IN THE APPS!





WHAT ARE THEY COLLECTING?

Restaurant reservations (Open Table)

Weather in L.A. in 3 days (Weather+)

Side effects of medications (MedWatcher)

3-star hotels in New Orleans (Priceline)

Which PC should I buy and where (PriceCheck)

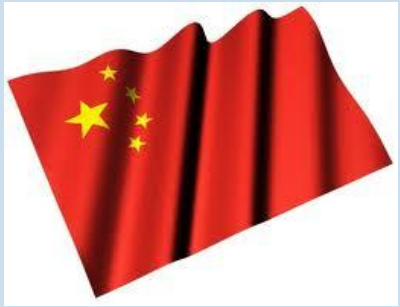
Who is collecting all of this data?

BIG PHARMACEUTICAL COMPANIES

GOVERNMENT AGENCIES



















Who is collecting all this data?

CONSUMER PRODUCTS COMPANIES





BIG BOX STORES

Sam's CLUB

THE HOME DEPOT

COSTCO WHOLESALE

STAPLES

KRAFT

WAL★MART
ALWAYS LOW PRICES.
Always.

Who is collecting what?

CREDIT CARD COMPANIES









WHAT DATA ARE THEY GETTING?

Airline ticket



Restaurant check



1	Cab Sauv Corvina	£15.25
1	Cab Sauv Glass	£4.00
20	Net Sales Total	£73.45
1	SUCK MY D[illegible] F[illegible] FACE	£0.00
1	FISH CAKES	£4.95
2	1/2 Wings Starter	£7.90
1	MELON AND PARMA HAM	£3.95
1	Calamari	£4.95
1	Garlic brd starter	£2.25
3	Meatball Starter	£17.85
1	Aub & Feta Starter	£3.95

Grocery Bill



Hotel Bill



Why are they collecting all this data?

TARGET MARKETING

To send you catalogs for exactly the merchandise you typically purchase.

To suggest medications that precisely match your medical history.

To “push” television channels to your set instead of your “pulling” them in.

To send advertisements on those channels **just for you**!

TARGETED INFORMATION

To know what you need before you even know you need it based on past purchasing habits!

To notify you of your expiring driver’s license or credit cards or last refill on a Rx, etc.

To give you turn-by-turn directions to a shelter in case of emergency.

Examples of *Big Data*

Global position satellite technology now allows trucking firms to track their trucks - and the merchandise inside them. Practically anything you can attach an ***RFID*** tag to can be tracked. How a company uses that information – to re-route trucks to create efficient routes, ***alert customers to deliveries***, and forecast and price services – depends on the ability to manage and analyze data effectively.

How much data?

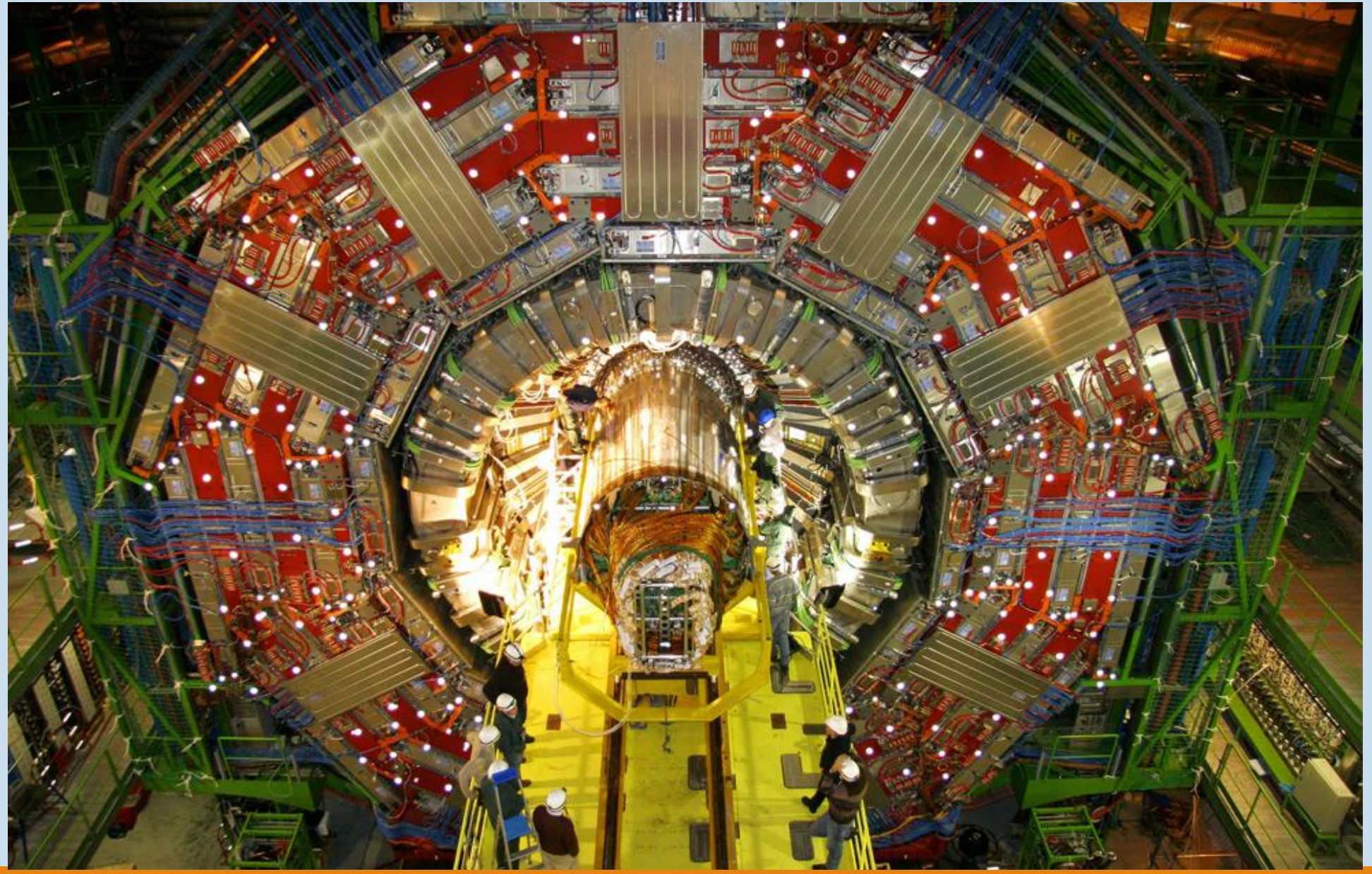
Google processes 20 PB a day (2008)

Wayback Machine has 3 PB + 100 TB/month (3/2009)

Facebook has 2.5 PB of user data + 15 TB/day (4/2009)

eBay has 6.5 PB of user data + 50 TB/day (5/2009)

CERN's Large Hydron Collider (LHC) generates 15 PB a year



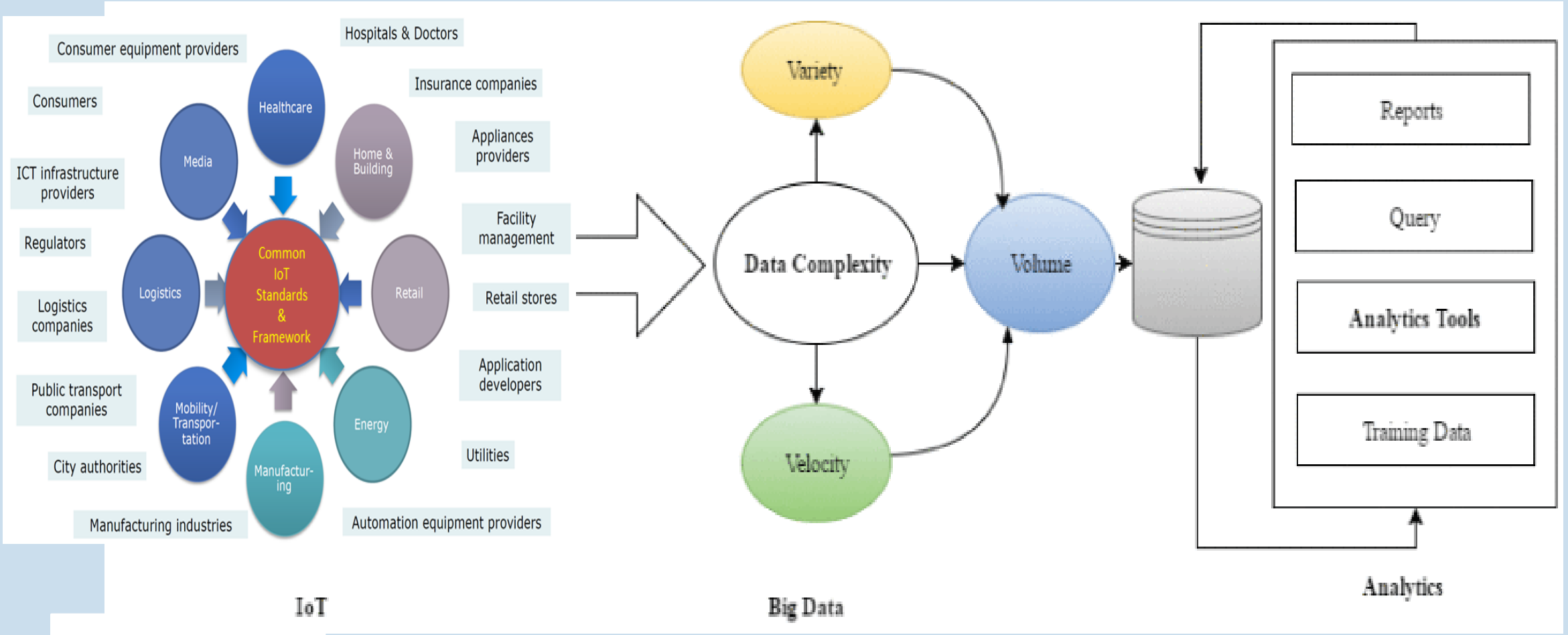
Maximilien Brice, © CERN

The Earthscope

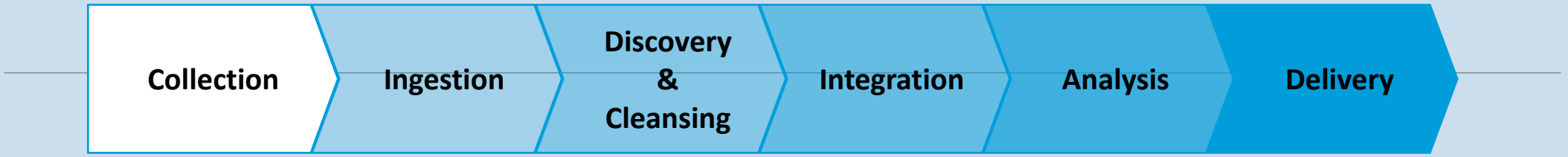
- The Earthscope is the world's largest science project. Designed to track North America's geological evolution, this observatory records data over 3.8 million square miles, amassing 67 terabytes of data. It analyzes seismic slips in the San Andreas fault, sure, but also the plume of magma underneath Yellowstone and much, much more. (http://www.msnbc.msn.com/id/44363598/ns/technology_and_science-future_of_technology/#.TmetOdQ--uI)



IOT Meets Big Data



Big Data Value Chain



Collection – Structured, unstructured and semi-structured data from multiple sources

Ingestion – loading vast amounts of data onto a single data store

Discovery & Cleansing – understanding format and content; clean up and formatting

Integration – linking, entity extraction, entity resolution, indexing and data fusion

Analysis – Intelligence, statistics, predictive and text analytics, machine learning

Delivery – querying, visualization, real time delivery on enterprise-class availability

Need for Standardized Approaches At Each Step

Organization

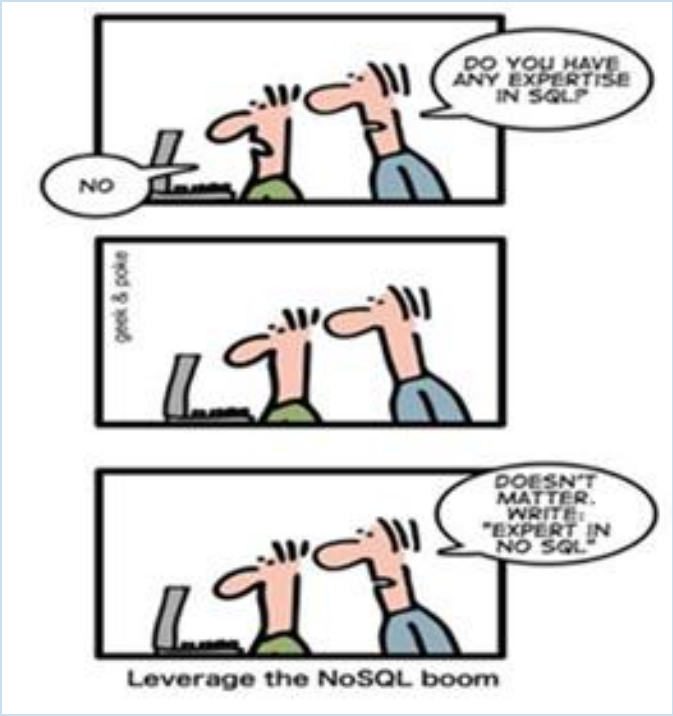
NoSQL Databases

To accommodate data with no schema and for *Real-Time Analytic Processing* **NoSQL** databases have been utilized.

NoSQL means ”not only SQL” or ”no SQL at all”.

NoSQL concept dates back to 1990s. They provide simpler scalability and improved performance comparing to traditional relational databases.

Data model is in NoSQL databases described intuitively without any formal fundamentals.



NoSQL Databases

In general NoSQL databases can be classified in the following groups:

- **key-value stores**: SimpleDB, Redis, Memcached, Dynamo, Voldemort
- **column-oriented**: BigTable, HBase, Hypertable, CASSANDRA, PNUTS
- **document-oriented**: MongoDB, CouchDB

To improve performance some NoSQL databases are **in-memory** databases (Redis and Memcached).

ACID properties are not implemented fully, databases can be only eventually consistent or weakly consistent.

List of NoSQL databases: http://nosql-database.org/

NewSQL Databases

A more general category of parallel DBMSs called NewSQL databases are designed to **scale out horizontally on shared nothing** machines while ensuring transparent partitioning and still guaranteeing **ACID** properties, and employing lock-free concurrency control.

Applications interact with the database primarily using **SQL**.

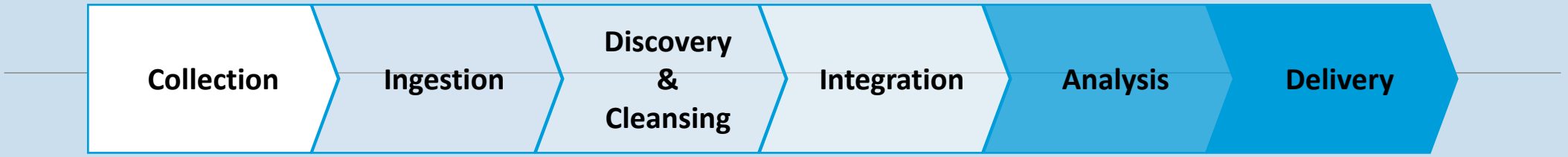
NewSQL provides performance and scalability not comparable with traditional DBMS and with Hadoop as well.

For example, a comparison of Hadoop and Vertica shown that query times for Hadoop were a lot slower (1-2 orders of magnitude).

In context of *Big Analytics*, most representatives of NewSQL are suitable for Real-time analytics (e.g., ClustrixDB, **Vertica, and VoltDB**) or columnar storage (Vertica).

But in general the **performance of NewSQL databases is still a problem.**

Big Data Value Chain



Collection – Structured, unstructured and semi-structured data from multiple sources

Ingestion – loading vast amounts of data onto a single data store

Discovery & Cleansing – understanding format and content; clean up and formatting

Integration – linking, entity extraction, entity resolution, indexing and data fusion

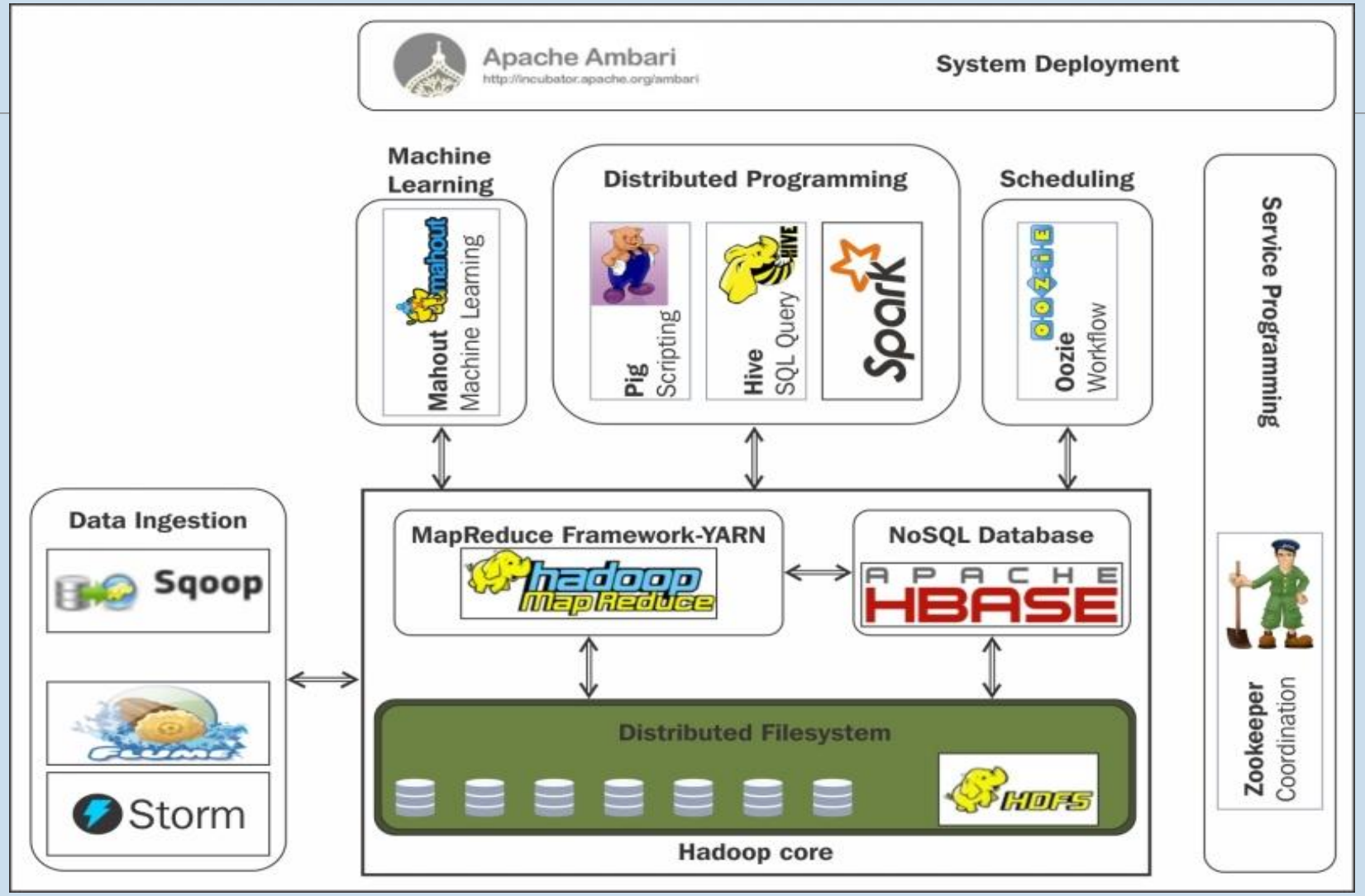
Analysis – Intelligence, statistics, predictive and text analytics, machine learning

Delivery – querying, visualization, real time delivery on enterprise-class availability

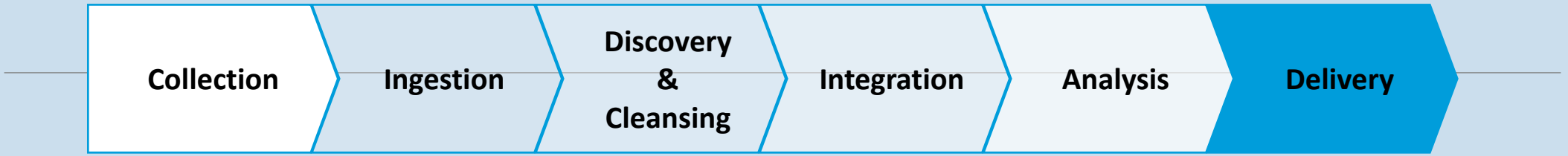
Need for Standardized Approaches At Each Step

Analysis

Hadoop Ecosystem



Big Data Value Chain



Collection – Structured, unstructured and semi-structured data from multiple sources

Ingestion – loading vast amounts of data onto a single data store

Discovery & Cleansing – understanding format and content; clean up and formatting

Integration – linking, entity extraction, entity resolution, indexing and data fusion

Analysis – Intelligence, statistics, predictive and text analytics, machine learning

Delivery – querying, visualization, real time delivery on enterprise-class availability

Need for Standardized Approaches At Each Step

Visualization

Traditional Visualization

Bar Charts, Pie Charts, line charts, bubble plots, scatter plots, heat maps, dot distribution maps, trees, matrices, etc….

Tools like tableau, plotly, charts.js, etc….

Big Data Visualization

Grafana – IOT visualization

Techniques

- Parallel Coordinates
- Star Coordinates

Hadoop

Hadoop History

Jan 2006 – Doug Cutting joins Yahoo

Feb 2006 – Hadoop splits out of Nutch and Yahoo starts using it.

Dec 2006 – Yahoo creating 100-node Webmap with Hadoop

Apr 2007 – Yahoo on 1000-node cluster

Jan 2008 – Hadoop made a top-level Apache project

Dec 2007 – Yahoo creating 1000-node Webmap with Hadoop

Sep 2008 – Hive added to Hadoop as a contrib project

Hadoop Introduction

Open Source Apache Project

- http://hadoop.apache.org/
- Book: http://oreilly.com/catalog/9780596521998/index.html

Written in Java

- Does work with other languages

Runs on

- Linux, Windows and more
- Commodity hardware with high failure rate

Open Source World's Solution

Google File System – *Hadoop Distributed FS*
Map-Reduce – *Hadoop Map-Reduce*
Sawzall – *Pig, Hive, JAQL*
Big Table – *Hadoop HBase*, *Cassandra*
Chubby – *Zookeeper*

Current Status of Hadoop

Largest Cluster

- 2000 nodes (8 cores, 4TB disk)

Used by 40+ companies / universities over the world

- Yahoo, Facebook, etc
- Cloud Computing Donation from Google and IBM

Startup focusing on providing services for hadoop

- Cloudera

Hadoop Components

Hadoop Distributed File System (HDFS)

Hadoop Map-Reduce

Contributes

- Hadoop Streaming
- Pig / JAQL / Hive
- HBase
- Hama / Mahout

Motivations for Hadoop

WHAT CONSIDERATIONS LED TO ITS DESIGN

Motivations for Hadoop

What were the limitations of earlier large-scale computing?

What requirements should an alternative approach have?

How does Hadoop address those requirements?

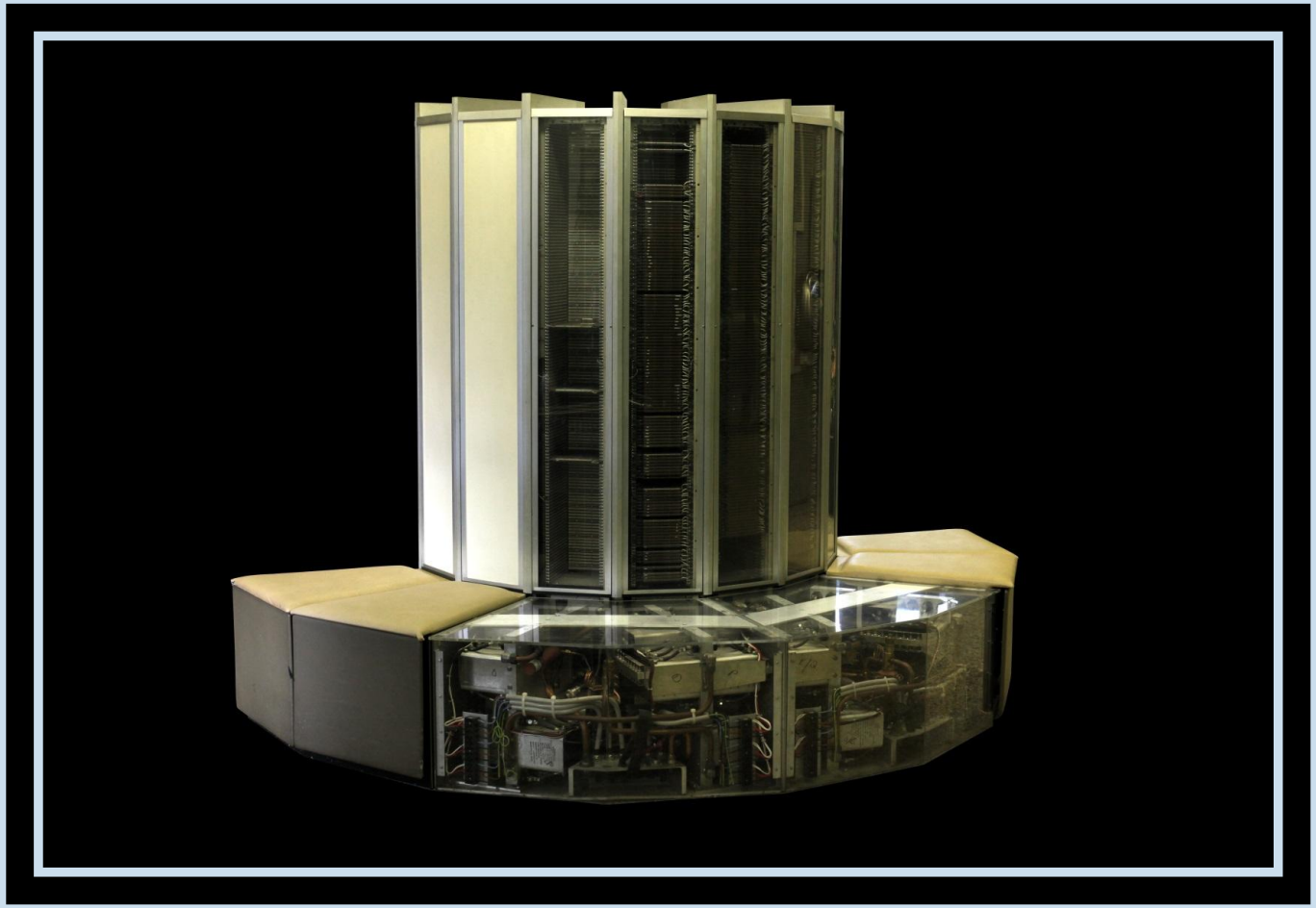
Early Large Scale Computing

Historically computation was processor-bound

- Data volume has been relatively small
- Complicated computations are performed on that data

Advances in computer technology has historically centered around improving the power of a single machine

Cray-1



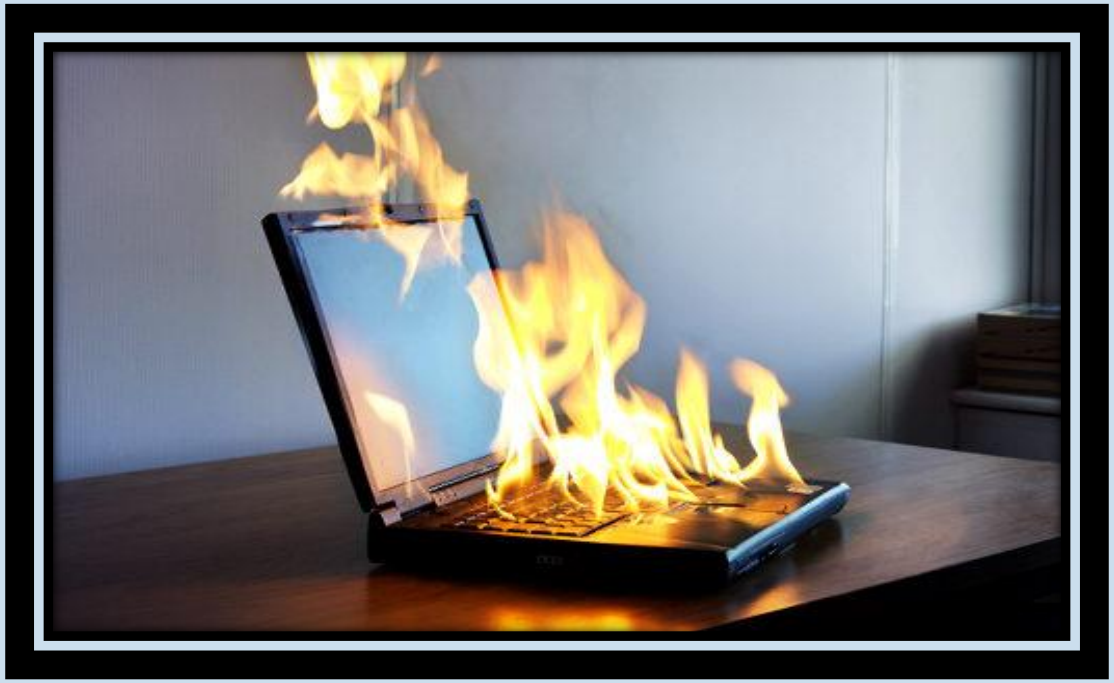
Advances in CPUs

Moore's Law
- The number of transistors on a dense integrated circuit doubles every two years

Single-core computing can't scale with current computing needs

Single-Core Limitation

Power consumption limits the speed increase we get from transistor density



Distributed Systems

Allows developers to use multiple machines for a single task



Distributed System: Problems

Programming on a distributed system is much more complex

- Synchronizing data exchanges
- Managing a finite bandwidth
- Controlling computation timing is complicated

Distributed System: Data Storage

Typically divided into Data Nodes and Compute Nodes

At compute time, data is copied to the Compute Nodes

Fine for relatively small amounts of data

Modern systems deal with far more data than was gathering in the past

How much data?

Facebook
- 500 TB per day

Yahoo
- Over 170 PB

eBay
- Over 6 PB

Getting the data to the processors becomes the bottleneck

Requirements for Hadoop

- Must support partial failure
- Must be scalable



Partial Failures

Failure of a single component must not cause the failure of the entire system only a degradation of the application performance

- Failure should not result in the loss of any data



Component Recovery

If a component fails, it should be able to recover without restarting the entire system

Component failure or recovery during a job must not affect the final output

Scalability

Increasing resources should increase load capacity

Increasing the load on the system should result in a graceful decline in performance for all jobs

- Not system failure

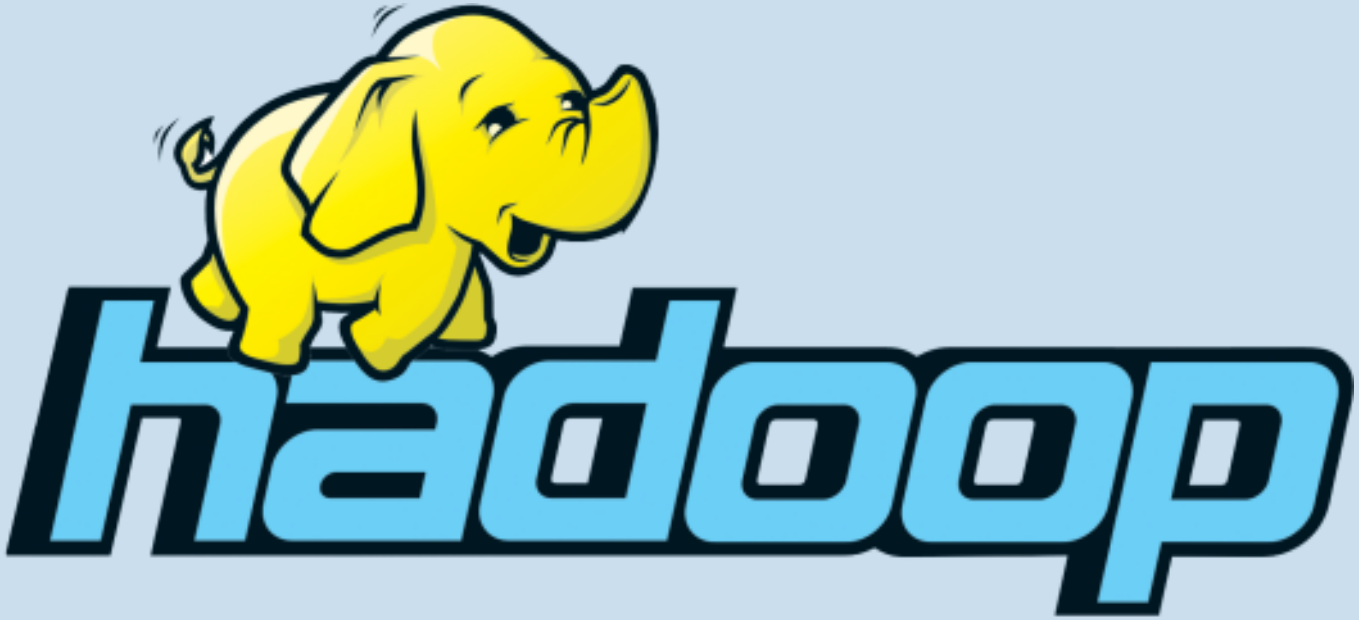
Hadoop

Based on work done by Google in the early 2000s

- "The Google File System" in 2003
- "MapReduce: Simplified Data Processing on Large Clusters" in 2004

The core idea was to distribute the data as it is initially stored

- Each node can then perform computation on the data it stores without moving the data for the initial processing


hadoop

Uses for Hadoop

Data-intensive text processing

Assembly of large genomes

Graph mining

Machine learning and data mining

Large scale social network analysis

Who Uses Hadoop?



The Hadoop Components

Hadoop Common	• Contains Libraries and other modules
HDFS	• Hadoop Distributed File System
Hadoop YARN	• Yet Another Resource Negotiator
Hadoop MapReduce	• A programming model for large scale data processing

HDFS User Interface

Java API

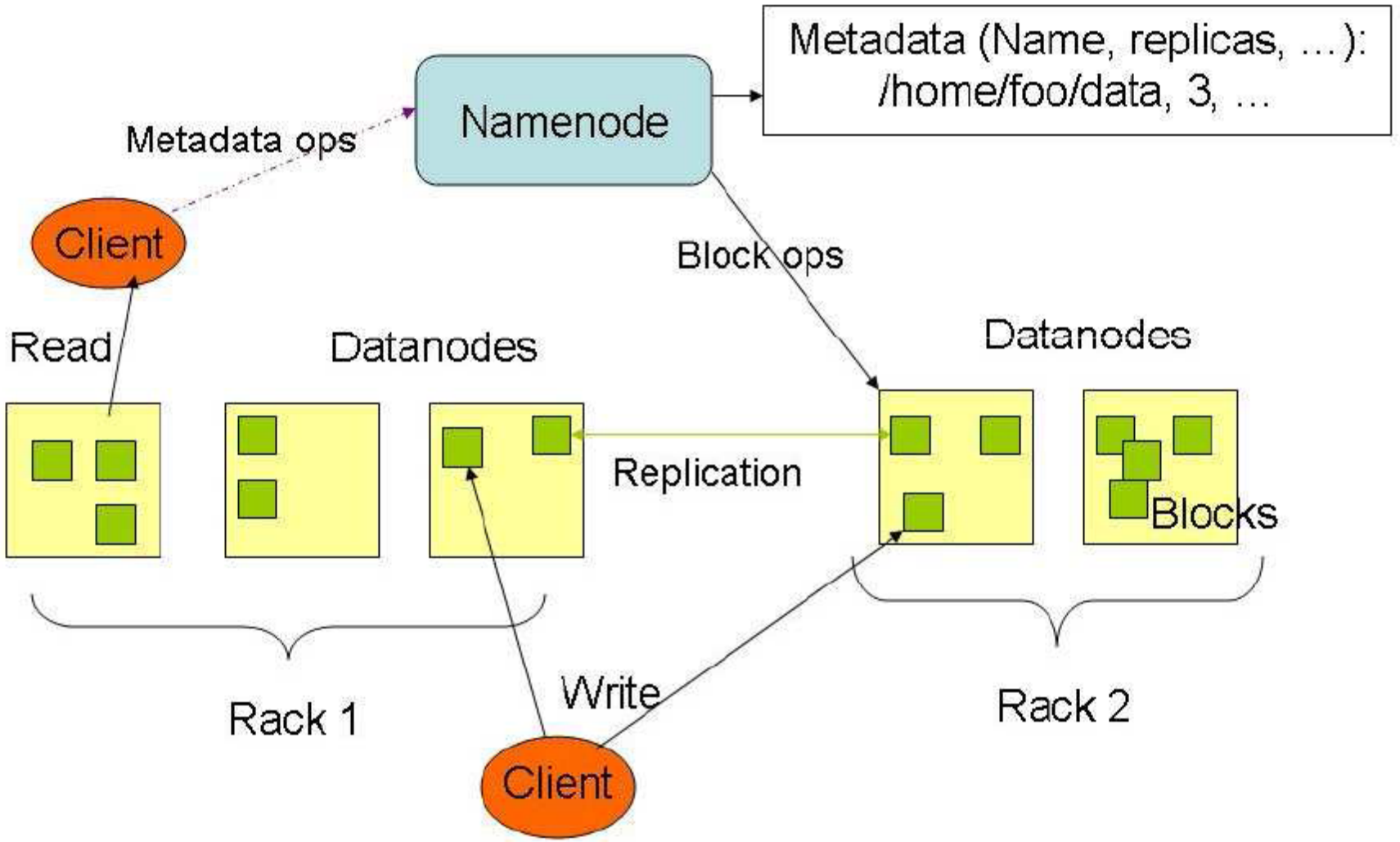
Command Line

- hadoop dfs -mkdir /foodir
- hadoop dfs -cat /foodir/myfile.txt
- hadoop dfs -rm /foodir myfile.txt
- hadoop dfsadmin -report
- hadoop dfsadmin -decommission datanodename

Web Interface

- http://host:port/dfshealth.jsp

HDFS Architecture



HDFS Details

Data Coherency

- Write-once-read-many access model
- Client can only append to existing files

Files are broken up into blocks

- Typically 128 MB block size
- Each block replicated on multiple DataNodes

Intelligent Client

- Client can find location of blocks
- Client accesses data directly from DataNode

Core Hadoop Concepts

Applications are written in a high-level programming language

- No network programming or temporal dependency

Nodes should communicate as little as possible

- A “shared nothing” architecture

Data is spread among the machines in advance

- Perform computation where the data is already stored as often as possible

High-Level Overview

When data is loaded onto the system it is divided into blocks

- Typically 64MB or 128MB

Tasks are divided into two phases

- Map tasks which are done on small portions of data where the data is stored
- Reduce tasks which combine data to produce the final output

A master program allocates work to individual nodes

Fault Tolerance

Failures are detected by the master program which reassigns the work to a different node

Restarting a task does not affect the nodes working on other portions of the data

If a failed node restarts, it is added back to the system and assigned new tasks

The master can redundantly execute the same task to avoid slow running nodes

Hadoop Distributed File System

HDFS

Overview

Responsible for storing data on the cluster

Data files are split into blocks and distributed across the nodes in the cluster

Each block is replicated multiple times

HDFS Basic Concepts

HDFS is a file system written in Java based on the Google's GFS

Provides redundant storage for massive amounts of data

HDFS Basic Concepts

HDFS works best with a smaller number of large files

- Millions as opposed to billions of files
- Typically 100MB or more per file

Files in HDFS are write once

Optimized for streaming reads of large files and not random reads

HDFS: Hadoop Distributed File System

HDFS contains the following key components:

NameNode:

- HDFS master node process
- manages the filesystem metadata
- does not store a file itself

SecondaryNameNode and Standby NameNode

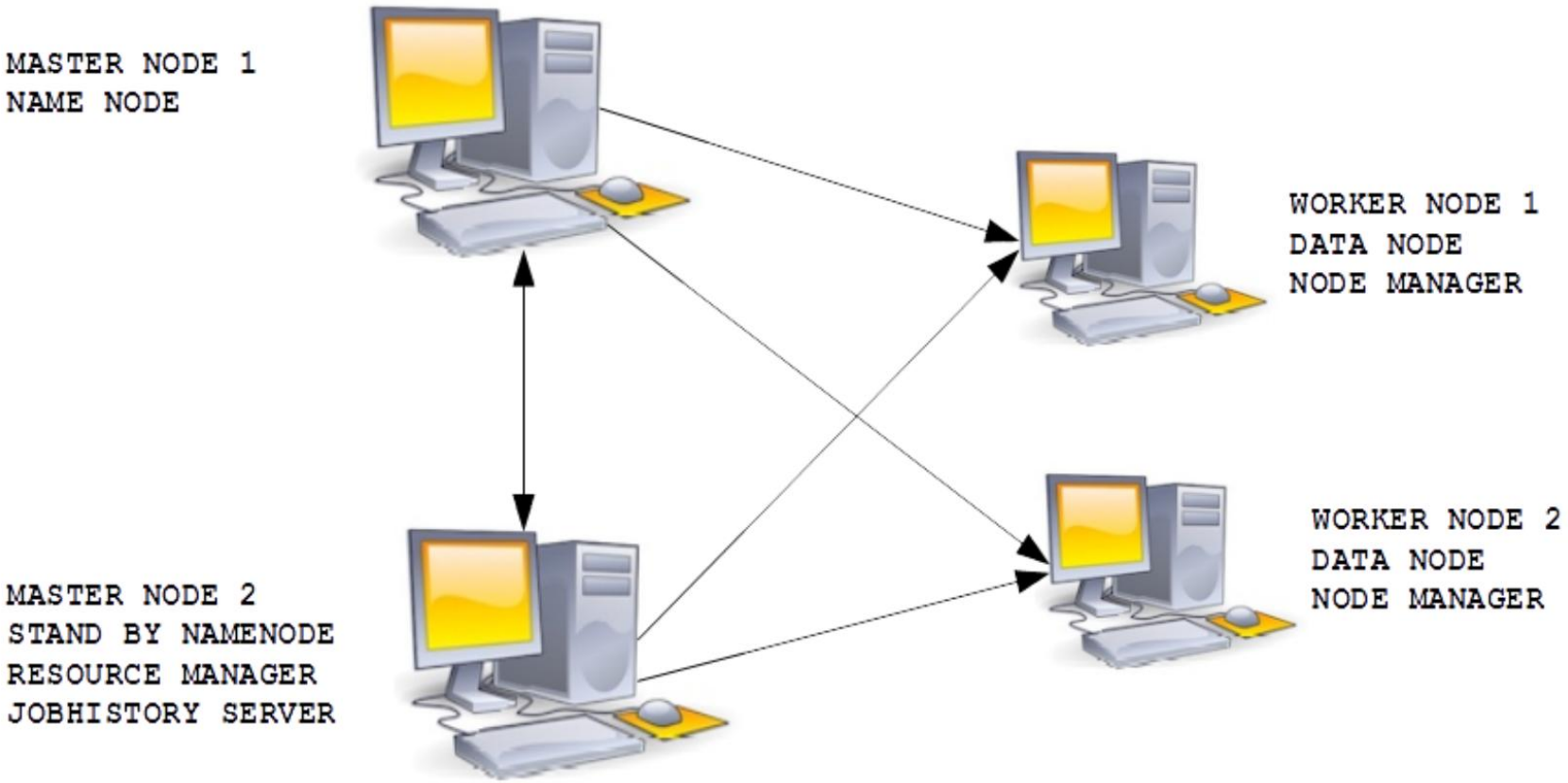
- SecondaryNameNode expedites the filesystem metadata recovery
- Standby NameNode (optional) provides high availability

DataNode

- runs HDFS slave node process
- manages block storage and access for reading or writing of data, block replication

HDFS: Hadoop Distributed File System

Architecture of HDFS



HDFS: Hadoop Distributed File System

HDFS is a virtual filesystem

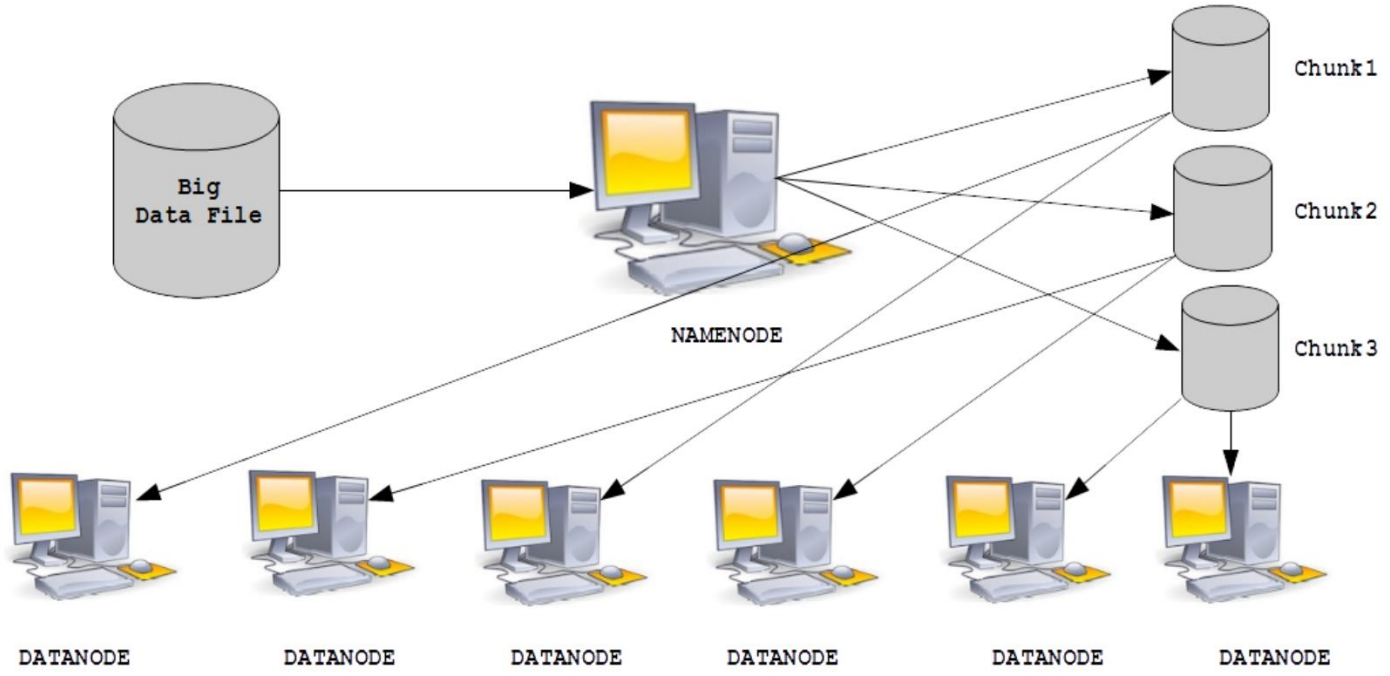
- appears to a client as one file system, but the data is stored in multiple different locations
- deployed on the top of the native filesystems (such as `ext3`, `ext4` and `xfs` in Linux)

Each file in HDFS consists of blocks

- The size of each block defaults to 128MB but is configurable
- The default number of replicates for blocks is 3, but it is also configurable

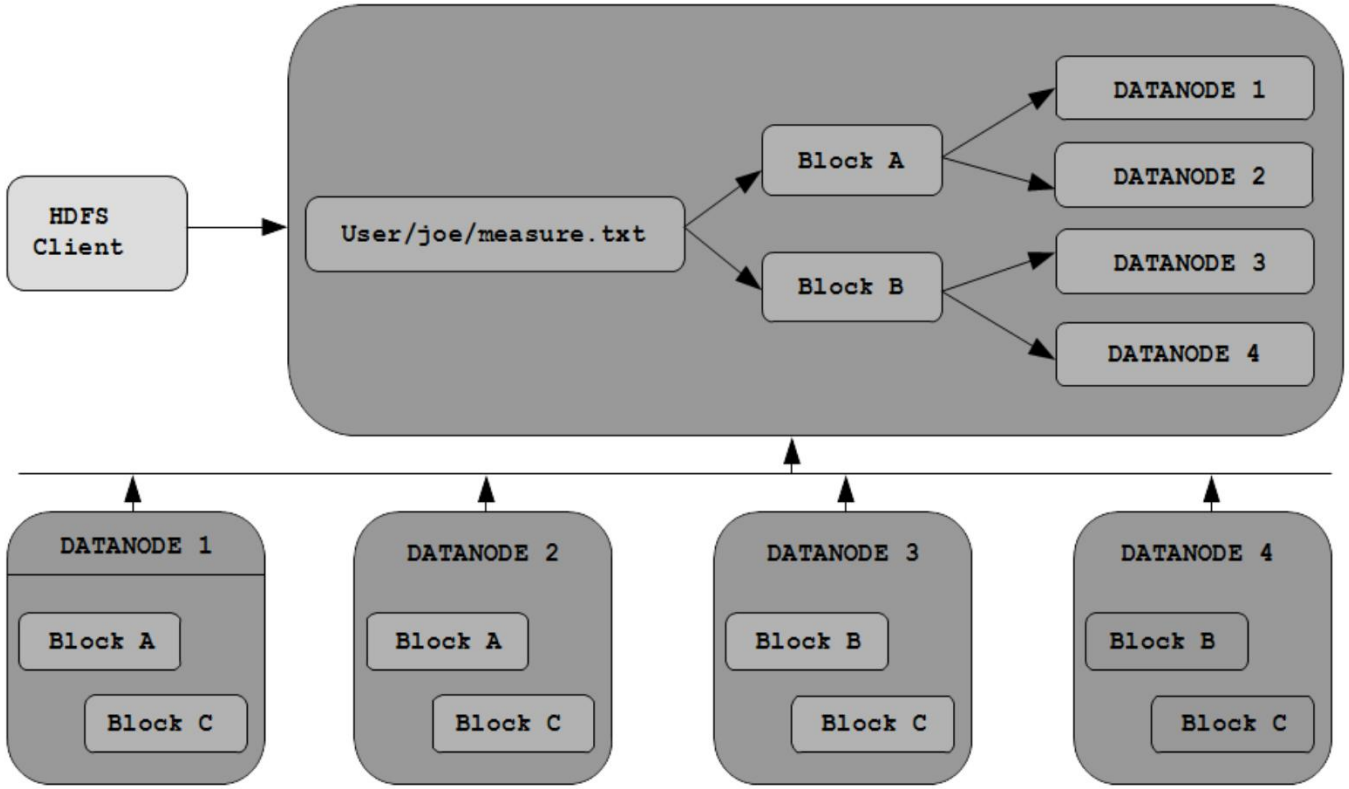
HDFS: Hadoop Distributed File System

Logical view of data storage



HDFS: Hadoop Distributed File Systeme

Physical implementation of data file storage

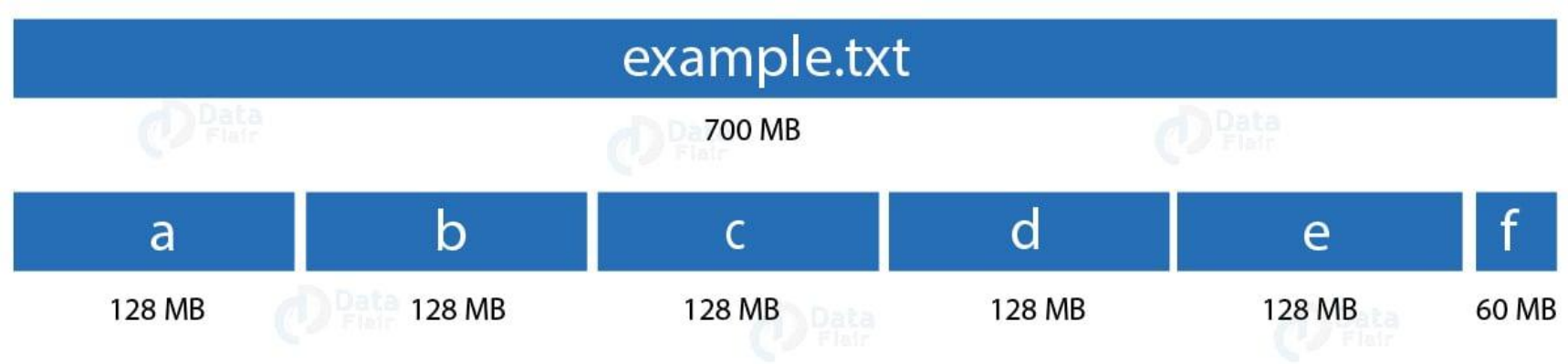


Hadoop Architecture

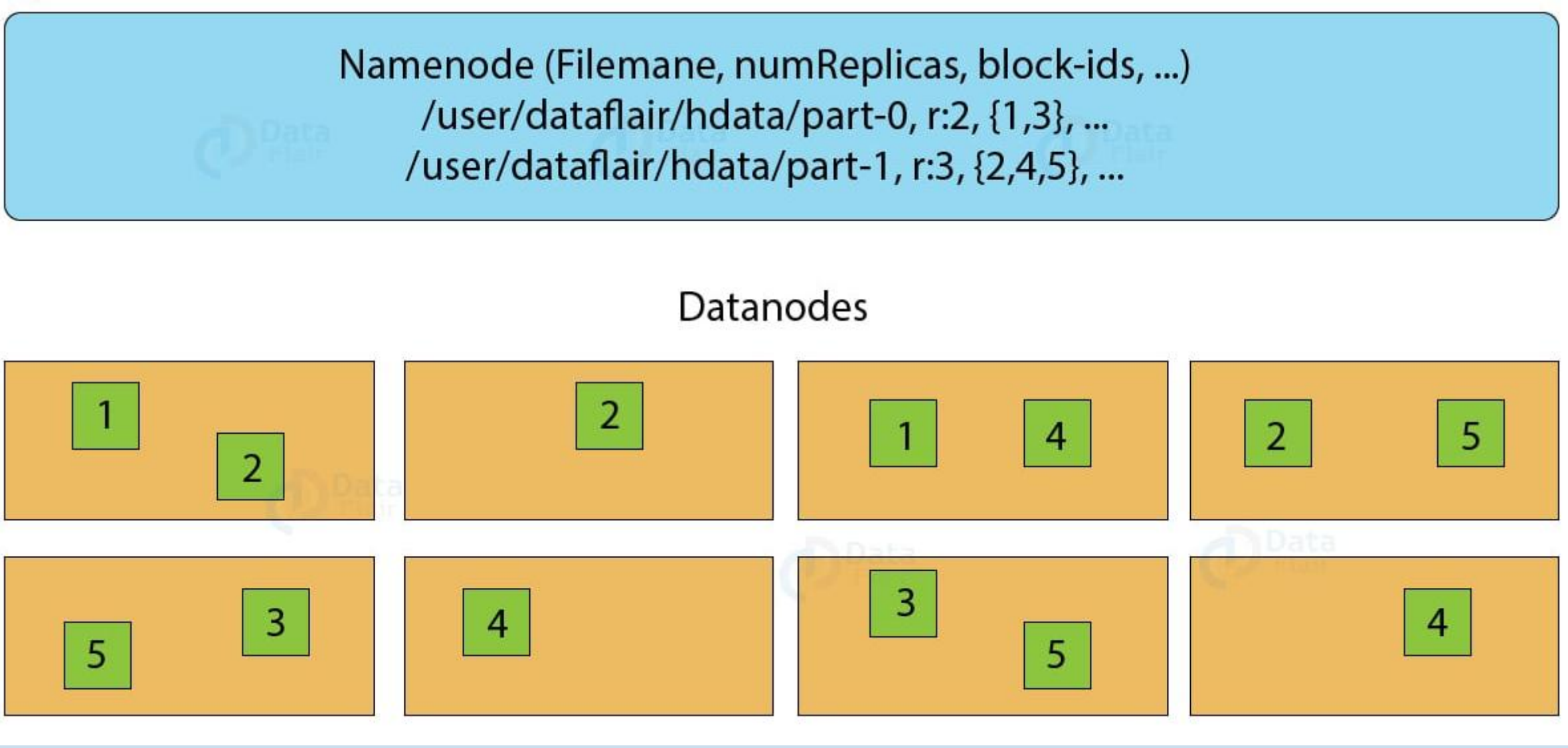
Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

Block is nothing but the smallest unit of storage on a computer system. It is the smallest contiguous storage allocated to a file. In Hadoop, **we have a default block size of 128MB or 256 MB**.



Block Replication



Rack Awareness

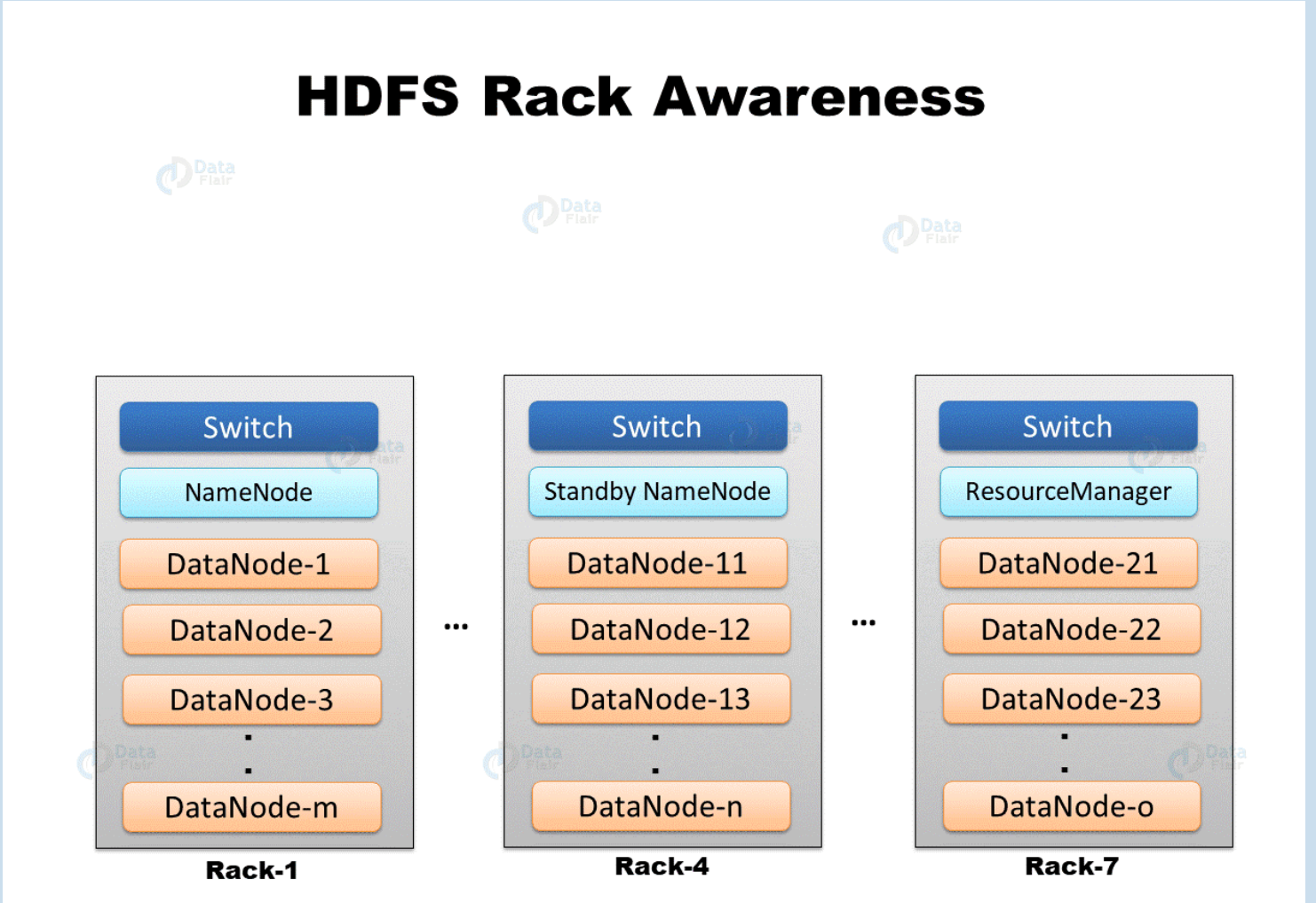
The reasons for the Rack Awareness in Hadoop are:

1.To reduce the network traffic while file read/write, which improves the cluster performance.

2.To achieve fault tolerance, even when the rack goes down (discussed later in this article).

3.Achieve high availability of data so that data is available even in unfavorable conditions.

4.To reduce the latency, that is, to make the file read/write operations done with lower delay.



Hadoop HDFS Data Read Operation

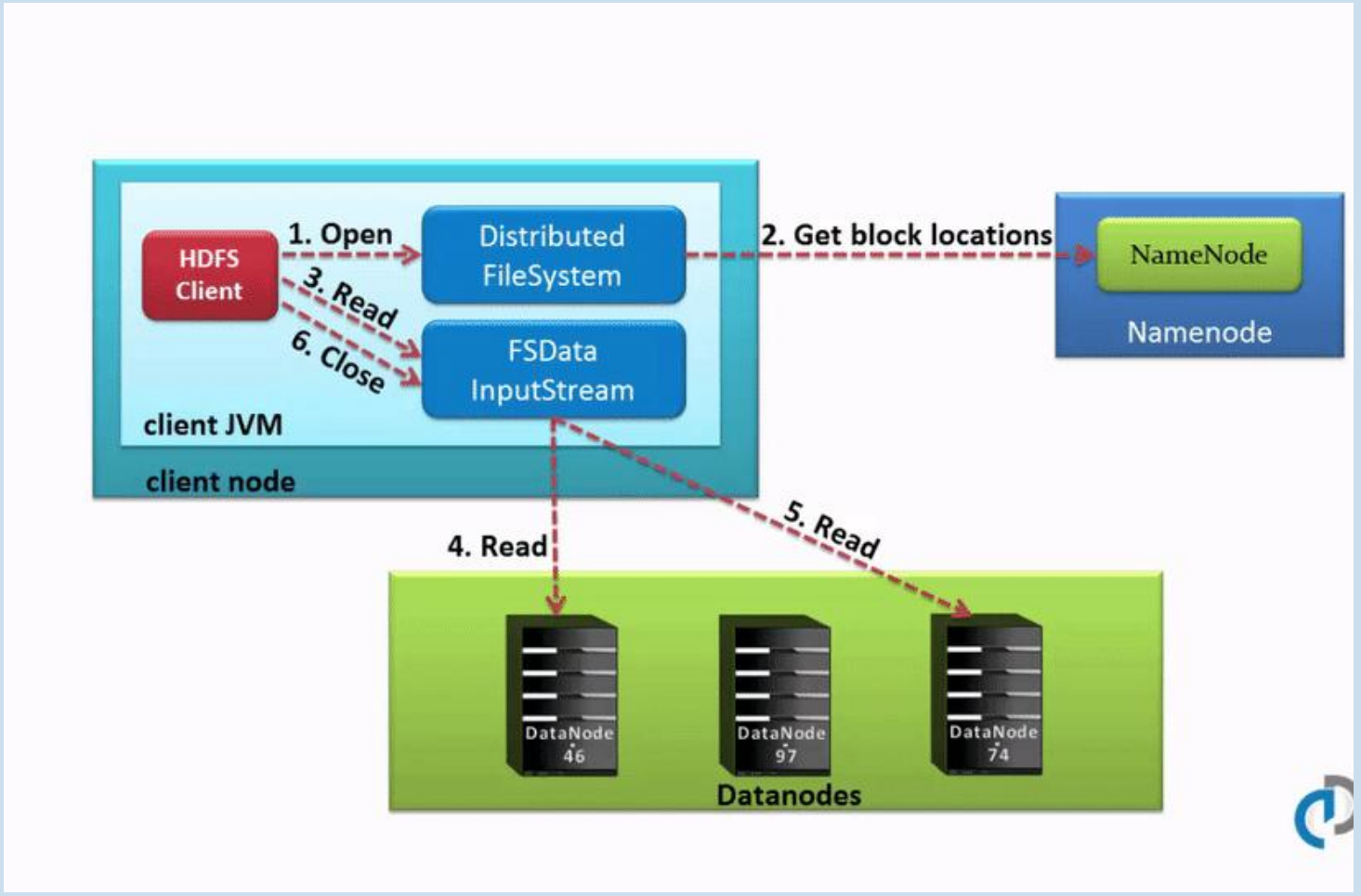
i) Client opens the file it wishes to read by calling **open()** on the *FileSystem* object, which for HDFS is an instance of *DistributedFileSystem*. See **Data Read Operation** in HDFS

ii) *DistributedFileSystem* calls the namenode using RPC to determine the locations of the blocks for the first few blocks in the file.

iii) *DistributedFileSystem* returns a *FSDataInputStream* to the client for it to read data from. *FSDataInputStream*, thus, wraps the *DFSInputStream* which manages the datanode and namenode I/O. Client calls **read()** on the stream.

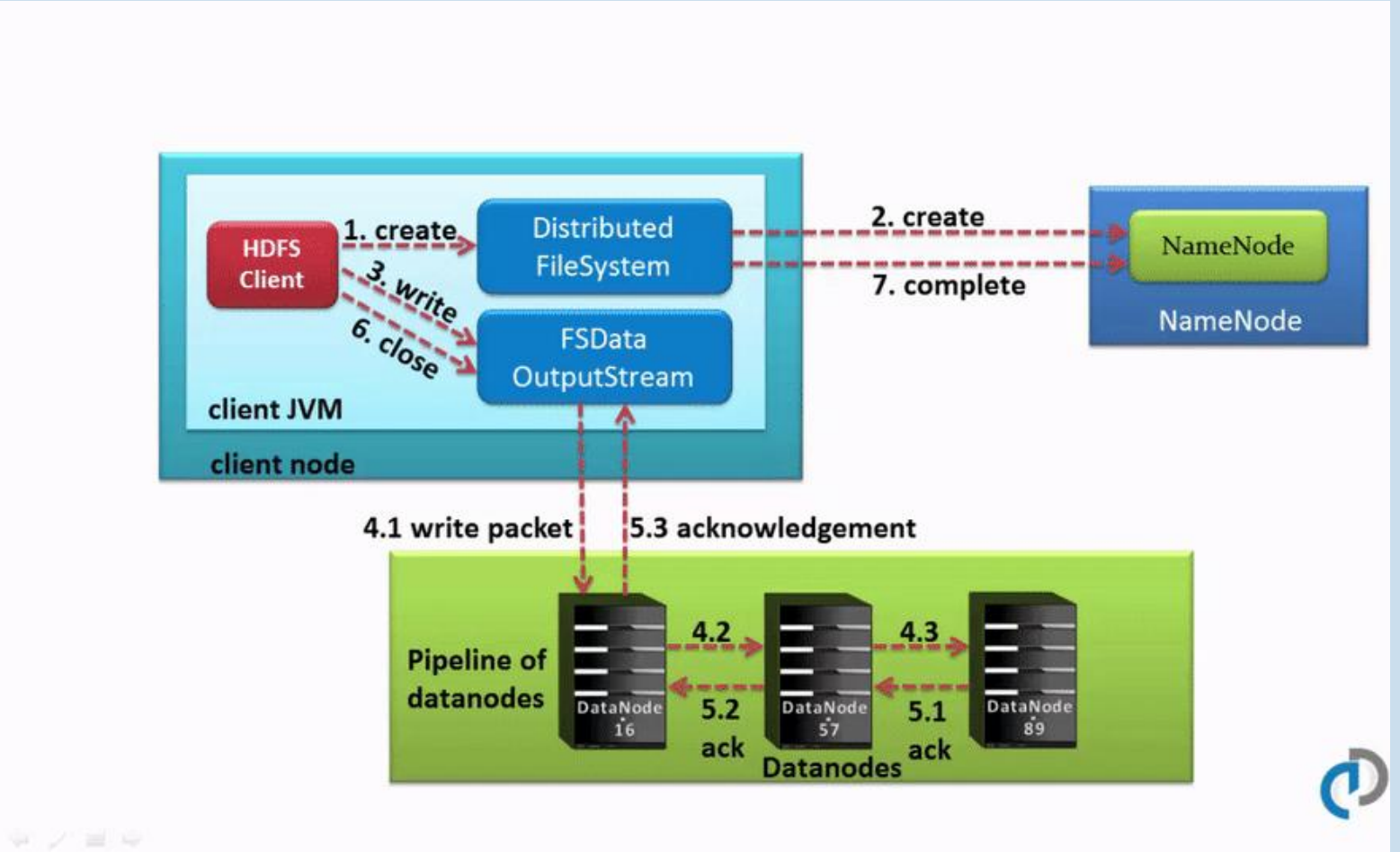
iv) Data is streamed from the datanode back to the client, as a result client can call **read()** repeatedly on the stream.

v) If the *DFSInputStream* encounters an error while communicating with a datanode, it will try the next closest one for that block.



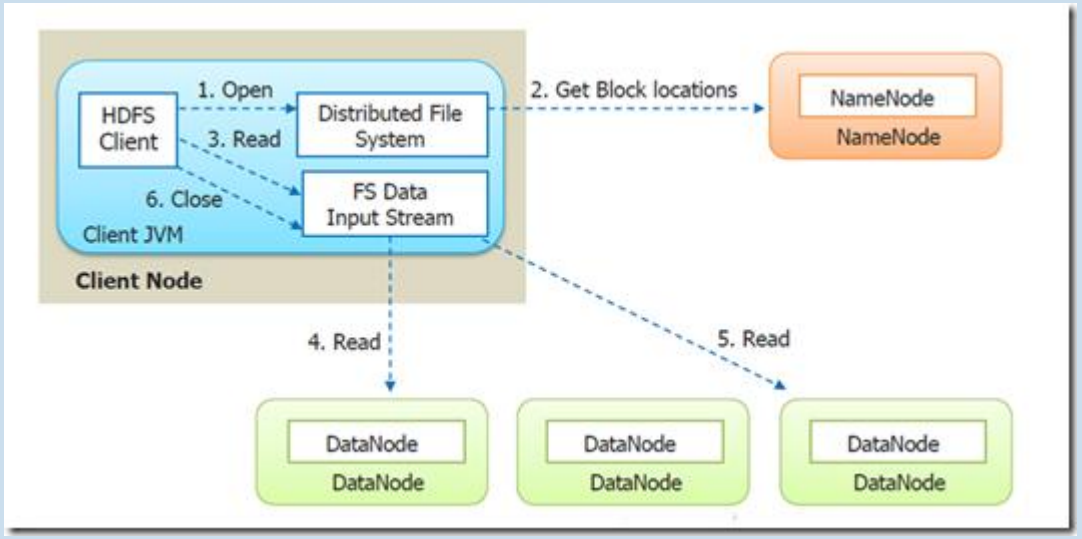
Hadoop HDFS Data Write Operation

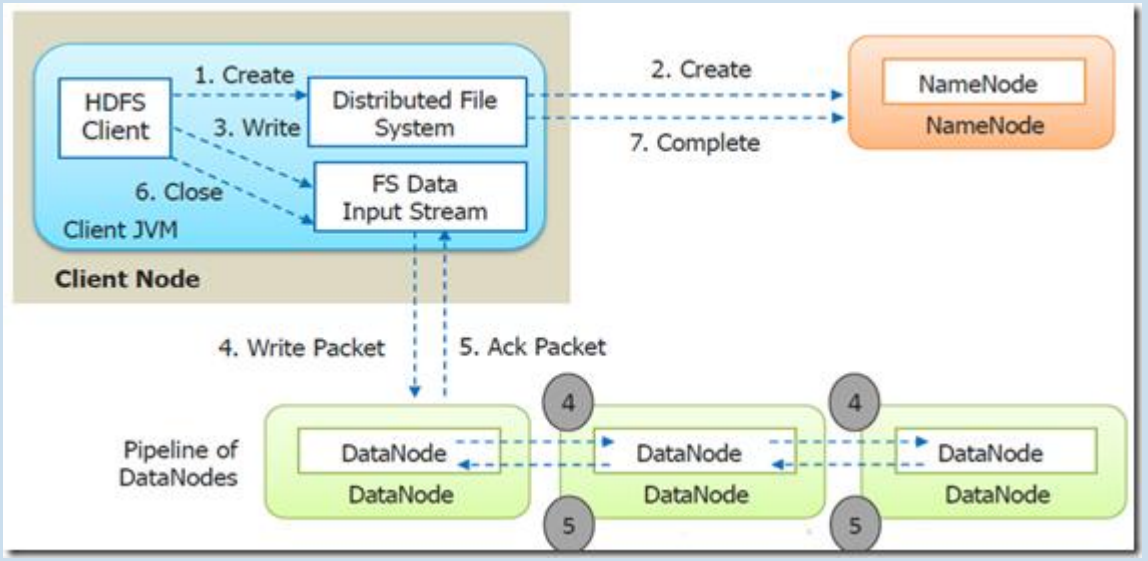
i) The HDFS client sends a **create** request on *DistributedFileSystem* APIs.
ii) *DistributedFileSystem* makes an RPC call to the namenode to create a new file in the file system's namespace.
iii) The *DistributedFileSystem* returns a *FSDataOutputStream* for the client to start writing data to.
iv) The list of datanodes form a pipeline, and here we'll assume the replication level is three, so there are three nodes in the pipeline.
v) *DFSOutputStream* also maintains an internal queue of packets that are waiting to be acknowledged by datanodes, called the *ack queue*.
vi) When the client has finished writing data, it calls **close()** on the stream.
vii) This action flushes all the remaining packets to the datanode pipeline and waits for acknowledgments before contacting the namenode to signal that the file is complete.



Advantages of HDFS Read/Write Design

- **Fault Tolerance:** Replication ensures data availability even if a DataNode fails.
- **Scalability:** Can handle petabytes of data across thousands of nodes.
- **High Throughput:** Optimized for batch processing and large dataset handling.
- **Data Locality:** Clients fetch data from the nearest DataNode to reduce latency.





NameNode Metadata

NameNode stores the metadata of the files in HDFS

object	block_id	seq	locations	ACL	Checksum
/data/file.txt	blk_00123	1	[node1,node2,node3]	-rwxrwxrwx	8743b52063..
/data/file.txt	blk_00124	2	[node2,node3,node4]	-rwxrwxrwx	cd84097a65..
/data/file.txt	blk_00125	3	[node2,node4,node5]	-rwxrwxrwx	d1633f5c74..

NameNode functions:

- Maintain the metadata pertaining to the file system (e.g., the file hierarchy and the block locations for each file)
- Manage user access to the data files
- Map the data blocks to the DataNodes in the cluster
- Perform file system operations (e.g., opening and closing the files and directories)
- Provide registration services and periodic heartbeats for DataNodes

Hadoop Architecture

Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

DataNode and Secondary node

DataNode functions:

- Provide the block storage by storing blocks on the local file system
- Fulfil the read/write requests
- Replicating data across the cluster
- Keeping in touch with the NameNode by sending periodic block reports and heartbeats
- A heartbeat confirms the DataNode is alive and healthy, and a block report shows the blocks being managed by the DataNode

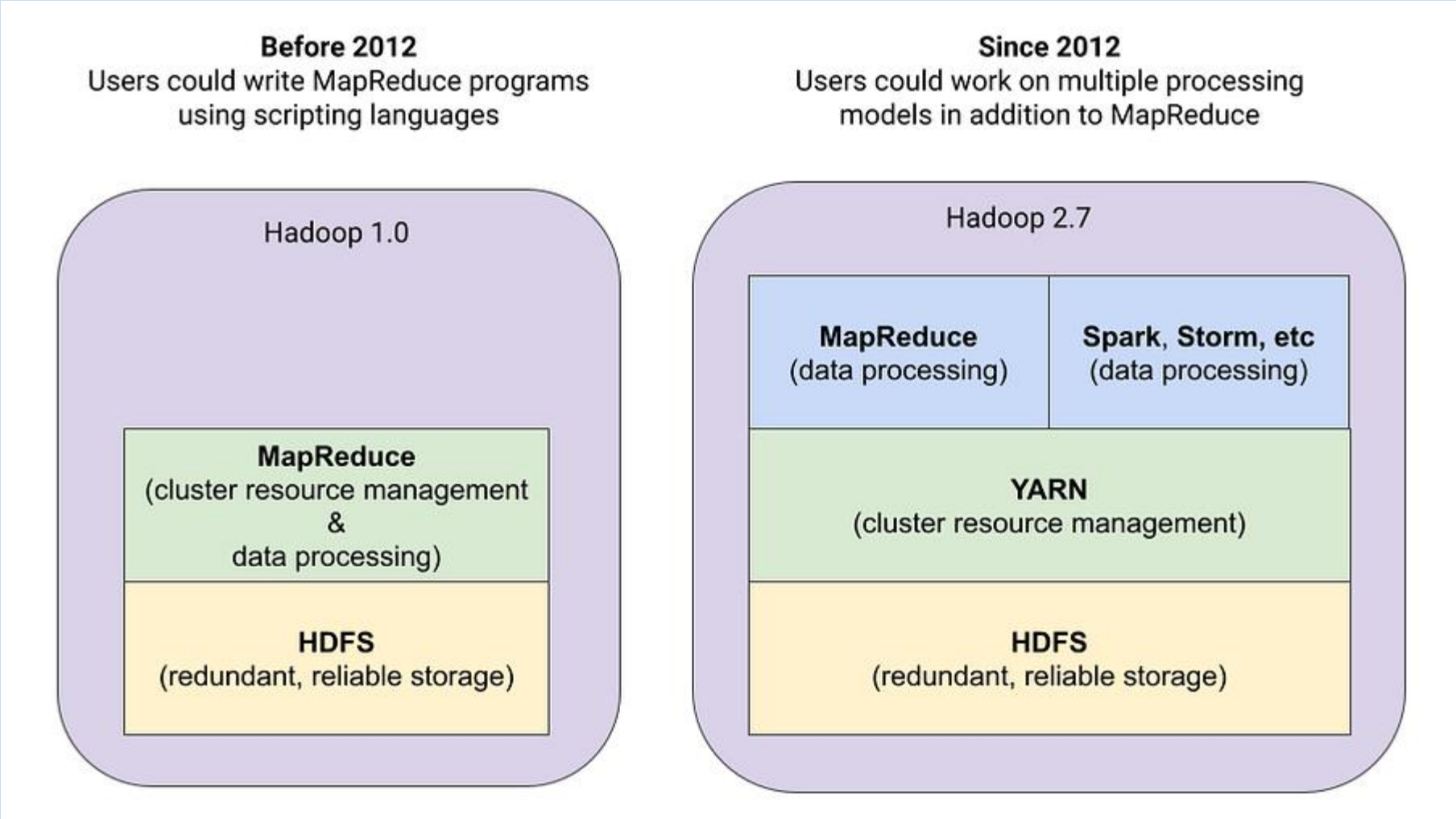
Secondary NameNode and Standby NameNode functions:

- Without a NameNode, there is no way to know to which files the blocks stored on the DataNodes correspond to
- In essence, all files in HDFS are lost
- Secondary NameNode periodically backups the metadata in the (primary) NameNode, which is usually for recovery
- Standby NameNode is a hot node that running together with the (primary) NameNode in the cluster, facilitating high-availability

Hadoop Architecture

Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary


Before 2012
Users could write MapReduce programs using scripting languages
Hadoop 1.0
MapReduce
(cluster resource management & data processing)
HDFS
(redundant, reliable storage)
Since 2012
Users could work on multiple processing models in addition to MapReduce
Hadoop 2.7
MapReduce
(data processing)
Spark, Storm, etc
(data processing)
YARN
(cluster resource management)
HDFS
(redundant, reliable storage)

Yet Another Resource Negotiator (YARN)

YARN: the core subsystem in Hadoop responsible for governing, allocating, and managing the finite distributed processing resources available on a Hadoop cluster

- introduced in Hadoop 2 to improve the MapReduce implementation, but general enough to support other distributed computing paradigms

YARN provides its core services via two types of long-running daemons:

- A ResourceManager (one per cluster) to manage the use of resources across the cluster, and
- NodeManagers running on all the nodes in the cluster to launch and monitor containers

Yet Another Resource Negotiator (YARN)

Architecture of YARN

A client is the program that submits jobs to the cluster

- May also be the gateway machine that the client program runs on

A job, also called an application, contains one or more tasks

- A task in a MapReduce job can be either a mapper and a reducer task

Each mapper and reducer task runs within a container

- Containers are logical constructs that represent a specific amount of memory and other resources, such as processing cores (CPU)
- For example, a container can represent 2GB memory and 2 processing cores
- Containers may also refer to the running environment of an application

Yet Another Resource Negotiator (YARN)

Architecture of YARN

ResourceManager: YARN's daemon running on a master node

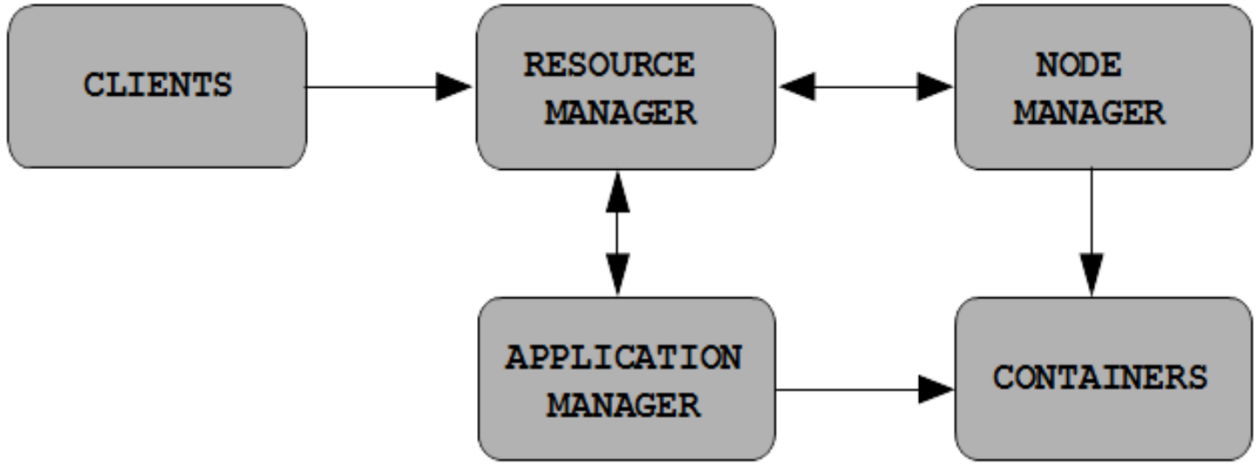
- ResourceManager is responsible for granting cluster computing resources to applications running on the cluster
- Resources are granted the items of containers

NodeManager: YARN's daemon running on a slave node.

- NodeManager manages containers on a slave node
- ApplicationMaster: the first container allocated by the ResourceManager to run on a NodeManager for each application

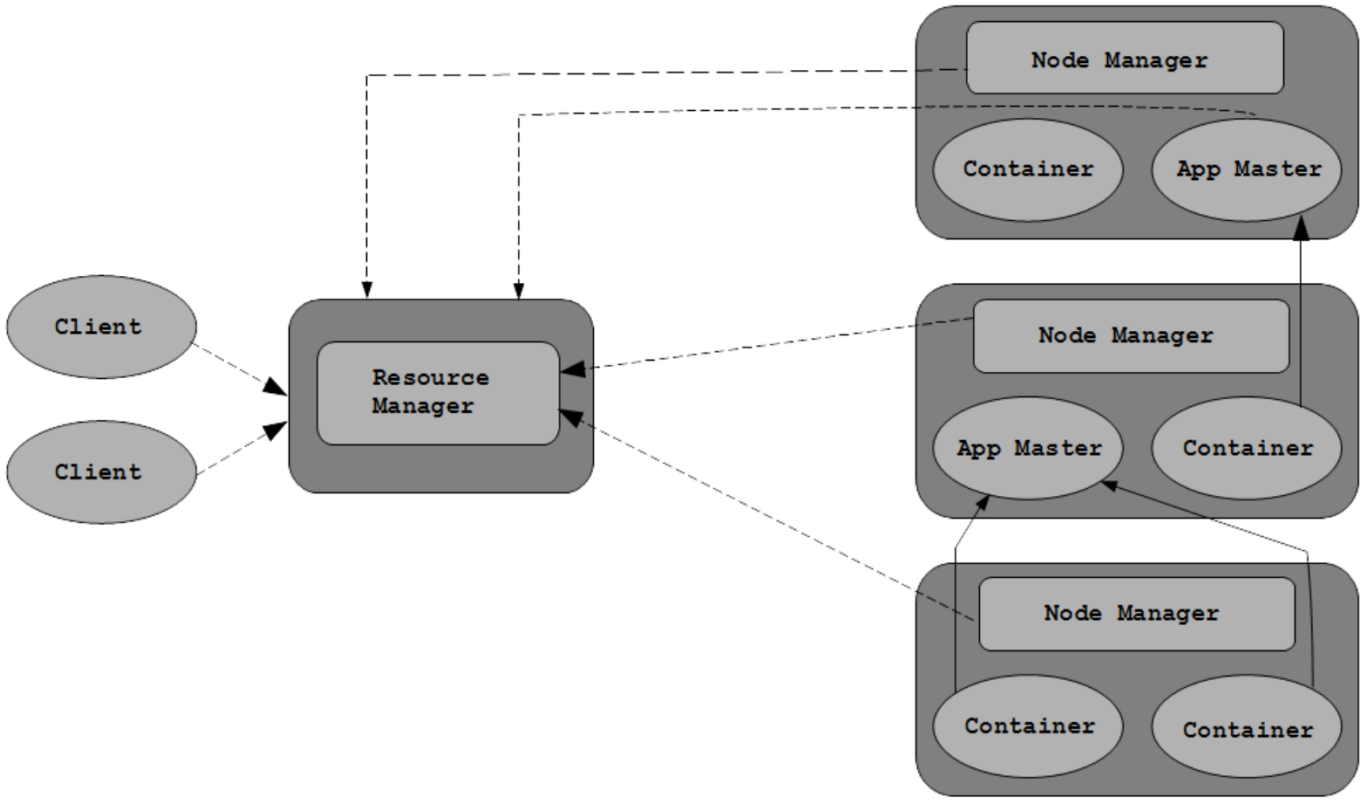
Yet Another Resource Negotiator (YARN)

Architecture of YARN



Yet Another Resource Negotiator (YARN)

Architecture of YARN



Hadoop Architecture

Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

ResourceManager

There is one ResourceManager per cluster, which consists of two key components: Scheduler and ApplicationManager

Key functions of ResourceManager:

- Creates the first container for an application to run ApplicationMaster for that application
- Tracks the heartbeats from NodeManagers to manage DataNodes
- Runs Scheduler to determine resource allocation among the clusters
- Manages cluster level security
- Manages the resource requests from ApplicationMasters
- Monitors the status of ApplicationMasters and restarts that container upon its failure
- Deallocates the containers when the application completes or after they expire

The role of ResourceManager is pure management and scheduler

It does not perform any actual data processing, for example the Map and Reduce functions in a MapReduce application

•**Scheduler**: Responsible for allocating resources to different applications based on defined policies and priorities. It uses resource requests (memory, CPU) and job scheduling constraints (e.g., fairness, capacity) to make resource decisions.

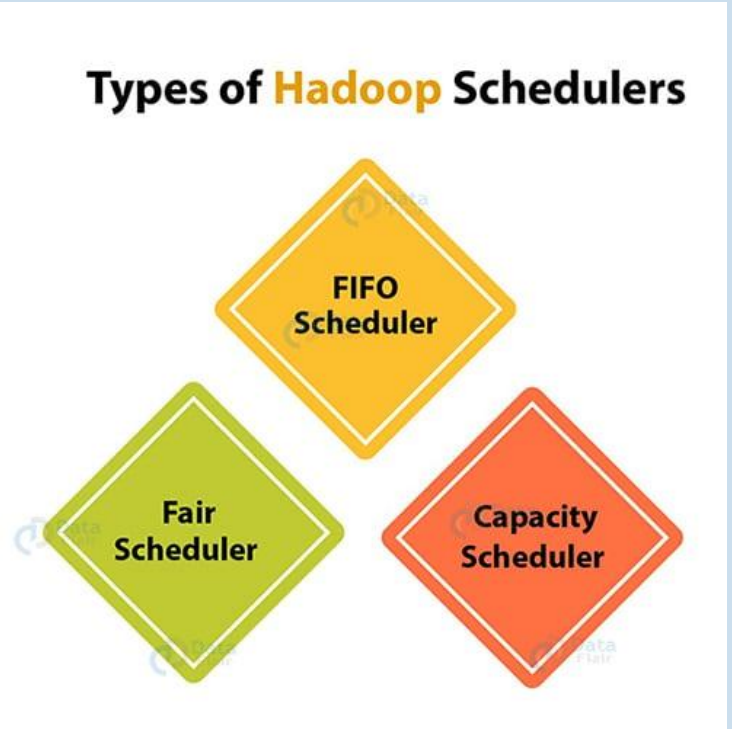
- Capacity Scheduler and Fair Scheduler are two common types of schedulers, each with different approaches to job allocation based on cluster priorities.

Key Technical Functions:

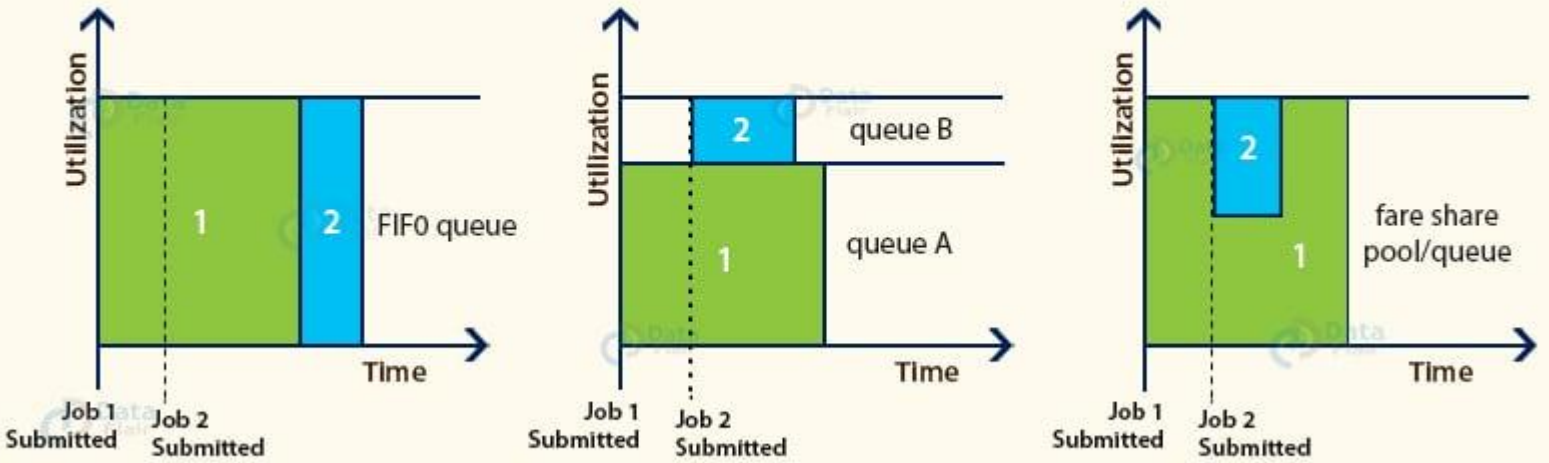
•Manages cluster-wide resource allocation and load balancing.

•Tracks cluster health, freeing resources from failed or completed tasks.

•Supports different types of job scheduling, including queue-based and resource-based policies.



Hadoop Schedulers



(a) FIFO Scheduler (b) Capacity Scheduler (c) Fair Scheduler

Hadoop Architecture

Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

NodeManager

Each DataNode runs a NodeManager daemon for performing YARN functions

Main functions of a NodeManager daemon:

- Communicates with ResourceManager through health heartbeats and container status notifications.
- Registers and starts the application processes
- Launches both ApplicationMaster and the rest of an application's resource containers (that is, the map and reduce tasks that run in the containers) on request from ApplicationMaster
- Oversees the lifecycle of the application containers
- Monitors, manages and provides information regarding the resource consumption (CPU/memory) by the containers
- Tracks the health of DataNode
- Provides auxiliary services to YARN applications, such as services used by the MapReduce framework for its shuffle and sort operations

Hadoop Architecture

Outline

- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

ApplicationMaster

For each YARN application, there is a dedicated ApplicationMaster

Functions of ApplicationMaster:

- Managing task scheduling and execution
- Allocating resources locally for the application's tasks

ApplicationMaster is running within a container

ApplicationMaster's existence is associated with the running application

When an application is completed, its ApplicationMaster no longer exists

Once created, ApplicationMaster is in charge of requesting resources with ResourceManager to run the application

The resource request are very specific, for example:

- the file blocks needed to process the job,
- the amount of the resource, in terms of the number of containers to create for the application,
- the size of the containers, etc.

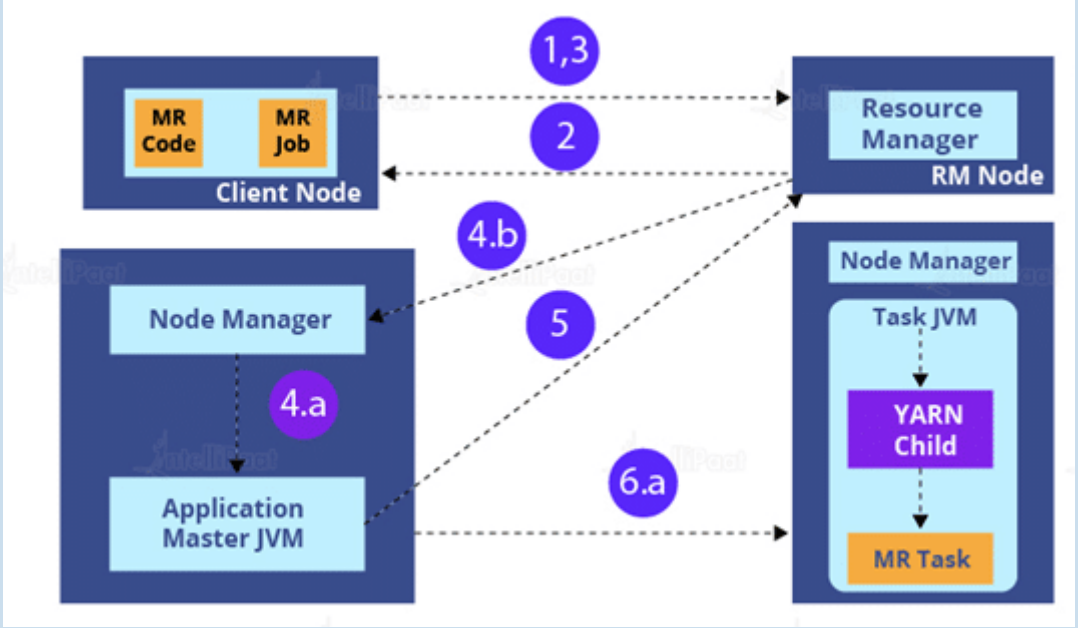
Hadoop Architecture

Outline

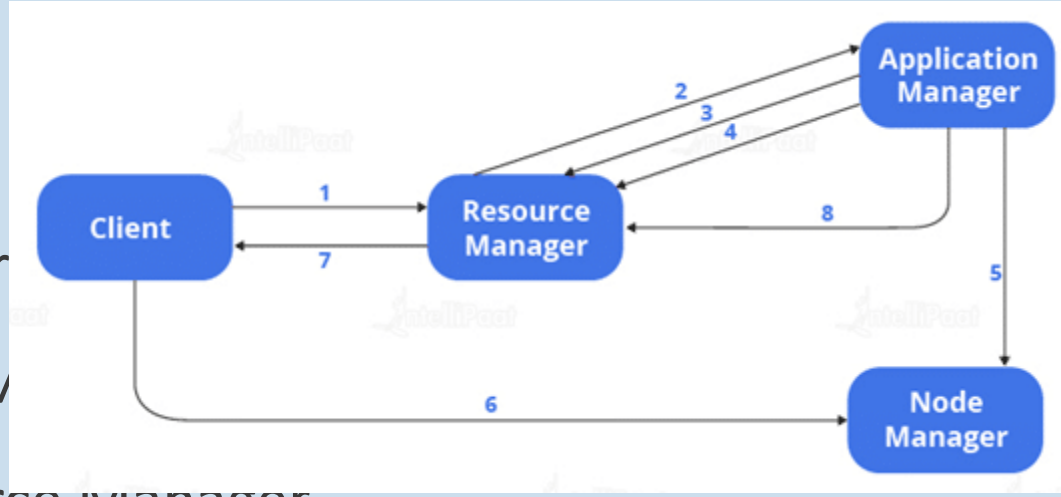
- Hadoop Distributed File System (HDFS)
- NameNode metadata
- DataNode and Secondary node
- Yet Another Resource Negotiator (YARN)
- ResourceManger
- NodeManager
- ApplicationMaster
- Summary

How is an application submitted in Hadoop YARN?

1. Submit the job
2. Get an application ID
3. Retrieval of the context of application submission
   - Start Container Launch
   - Launch Application Master
4. Allocate Resources.
   - Container
   - Launching
5. Executing



Workflow of an Application in Apache Hadoop YARN



1.Submission of the application by Client

2.Container allocation for starting Application Manager

3.Registering the Application Manager with Resource M

4.Application Manager asks for containers from Resource Manager

5.Application Manager notifies Node Manager to launch containers

6.Application code gets executed in the container

7.Client contacts Resource Manager/Application Manager to monitor the status of the application

8.Application Manager gets disconnected with Resource Manager

Features of Hadoop YARN

•High-degree compatibility: Applications created use the MapReduce framework that can be run easily on YARN.

•Better cluster utilization: YARN allocates all cluster resources efficiently and dynamically, which leads to better utilization of Hadoop as compared to the previous version of it.

•Utmost scalability: Whenever there is an increase in the number of nodes in the Hadoop cluster, the YARN Resource Manager assures that it meets the user requirements.

•Multi-tenancy: Various engines that access data on the Hadoop cluster can efficiently work together all because of YARN as it is a highly versatile technology.

MapReduce

DISTRIBUTING COMPUTATION ACROSS NODES

MapReduce Overview

A method for distributing computation across multiple nodes

Each node processes the data that is stored at that node

Consists of two main phases

- Map
- Reduce

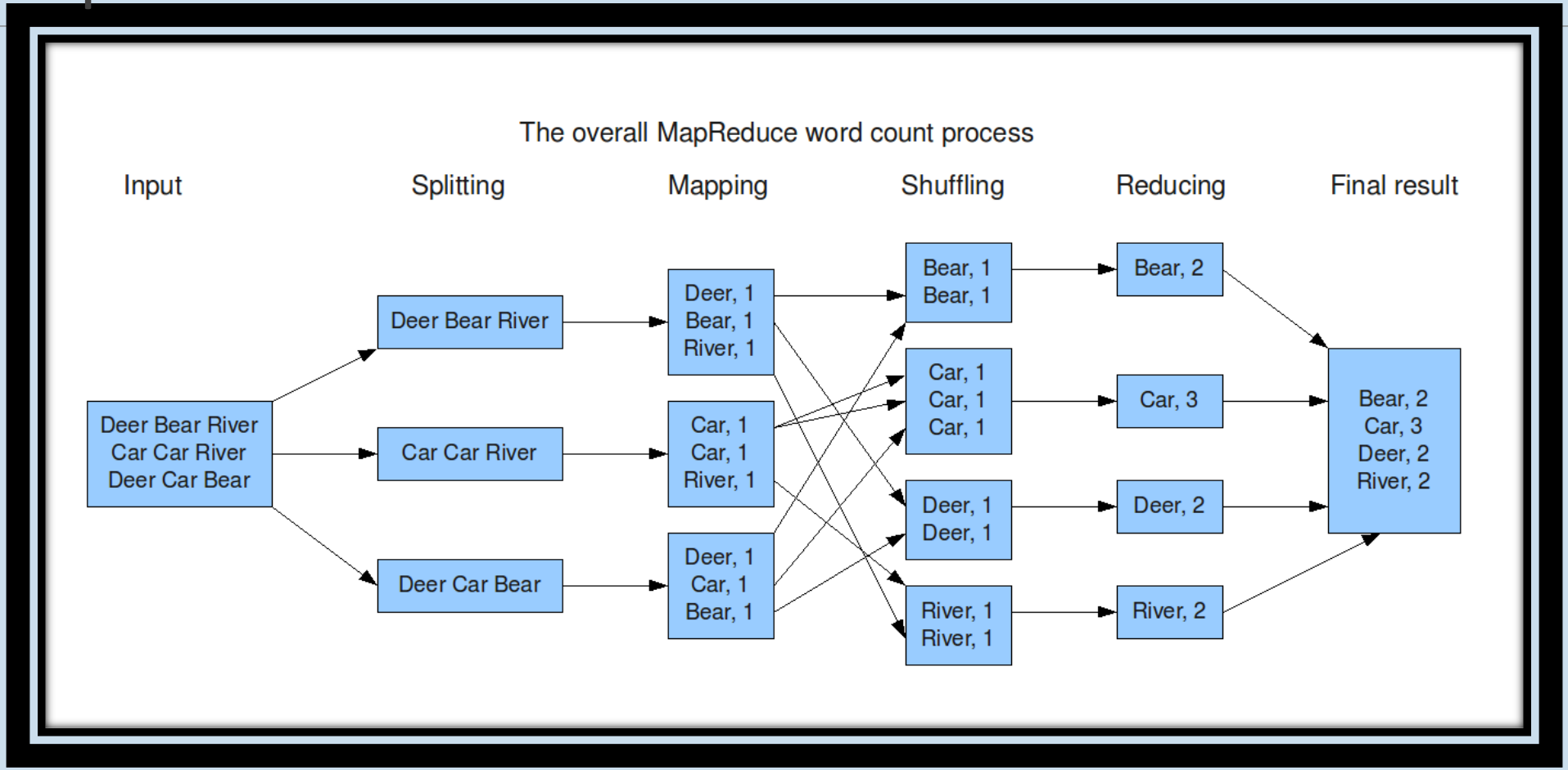
MapReduce Features

Automatic parallelization and distribution

Fault-Tolerance

Provides a clean abstraction for programmers to use

MapReduce: Word Count



The Mapper

Reads data as key/value pairs

- The key is often discarded

Outputs zero or more key/value pairs

Shuffle and Sort

Output from the mapper is sorted by key

All values with the same key are guaranteed to go to the same machine

The Reducer

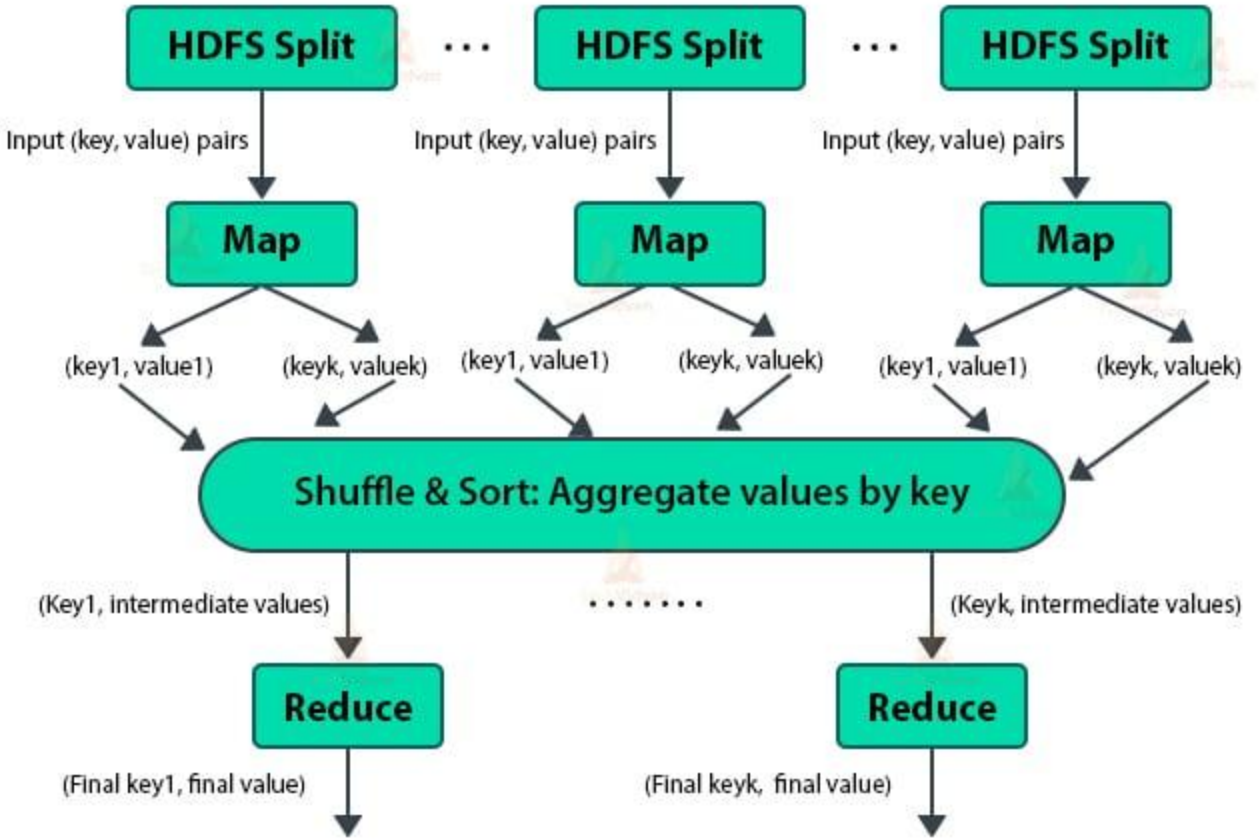
Called once for each unique key

Gets a list of all values associated with a key as input

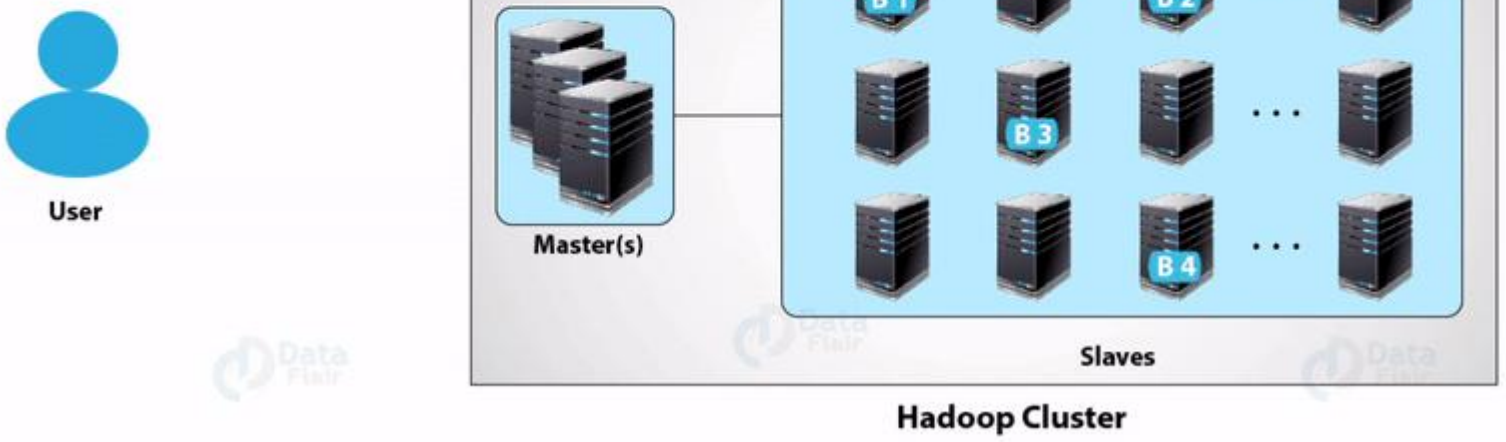
The reducer outputs zero or more final key/value pairs

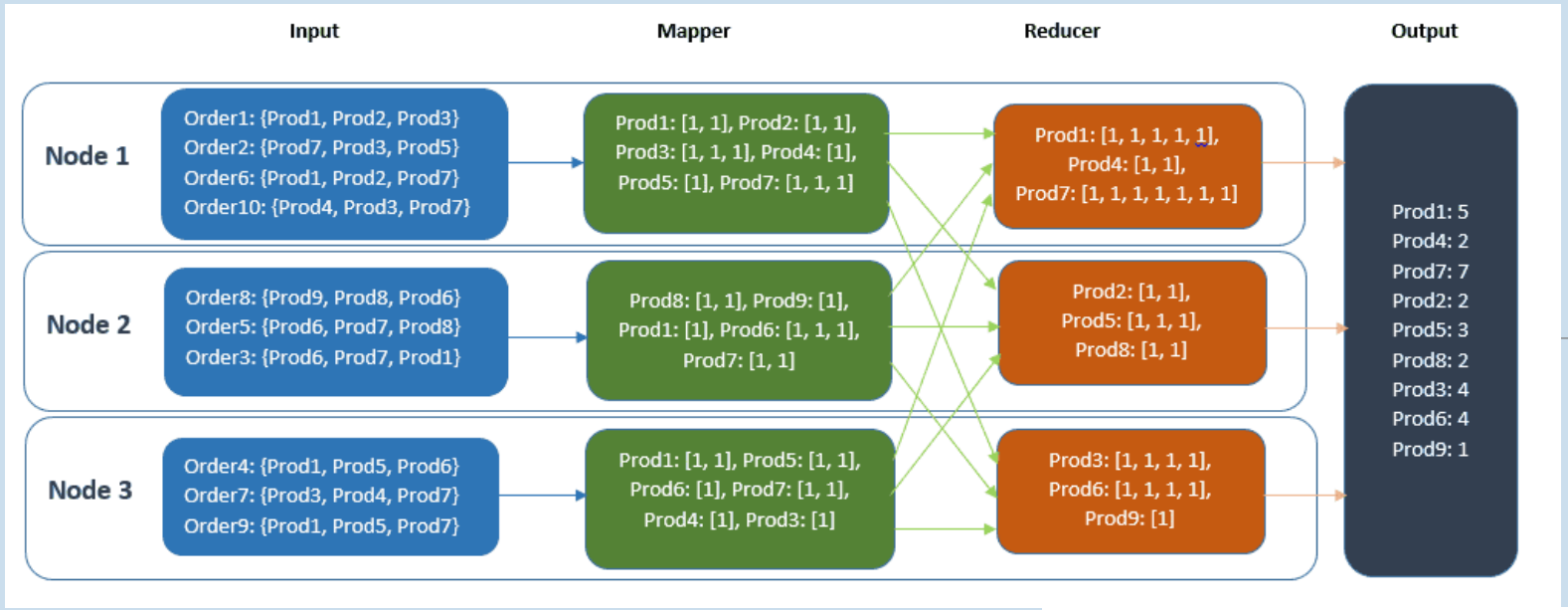
- Usually just one output per input key

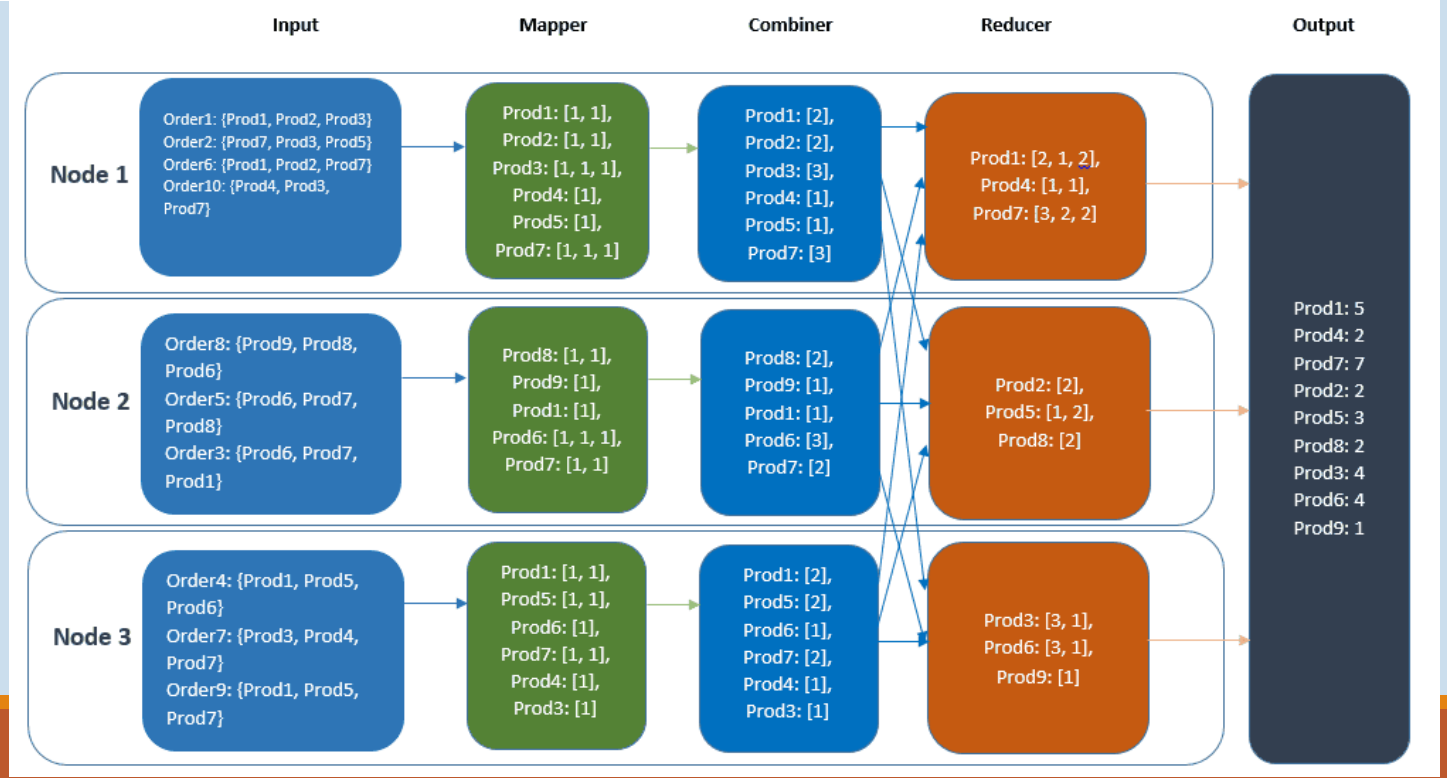
Hadoop MapReduce



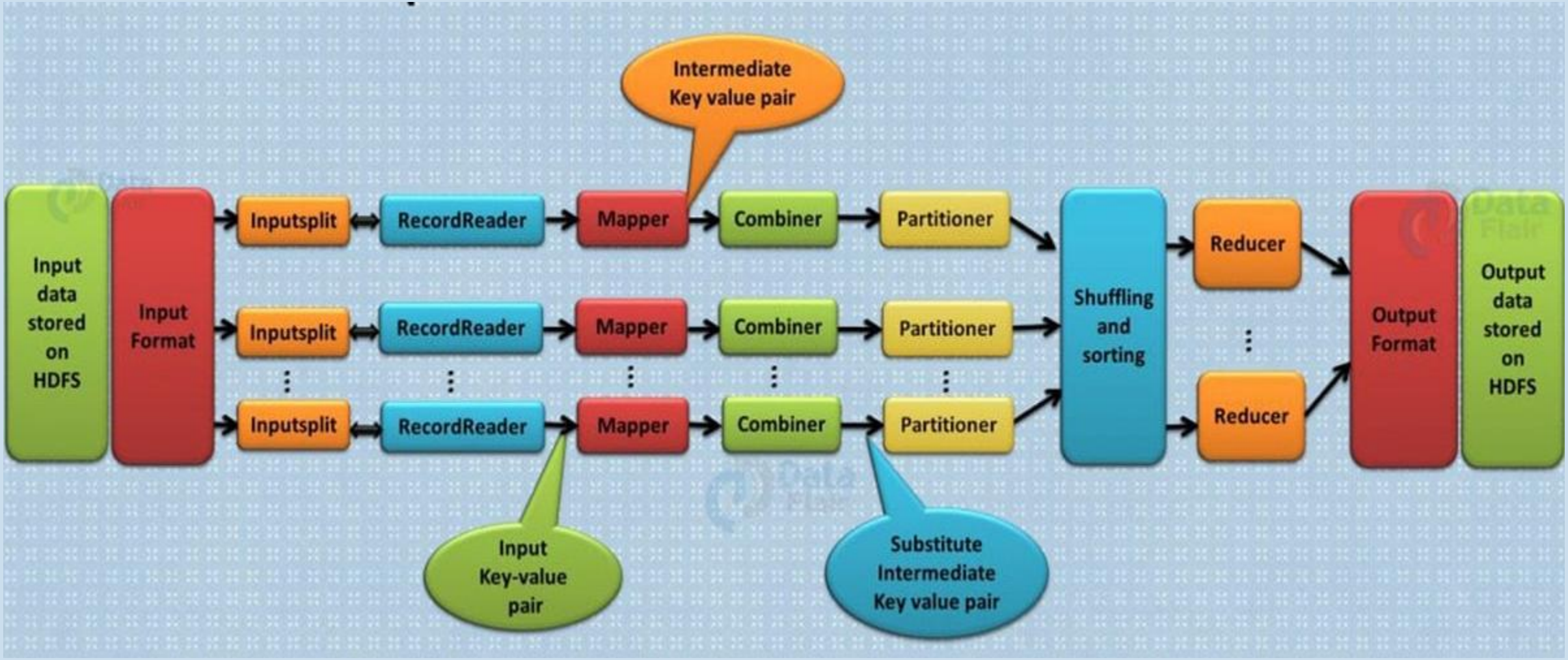
How MapReduce works

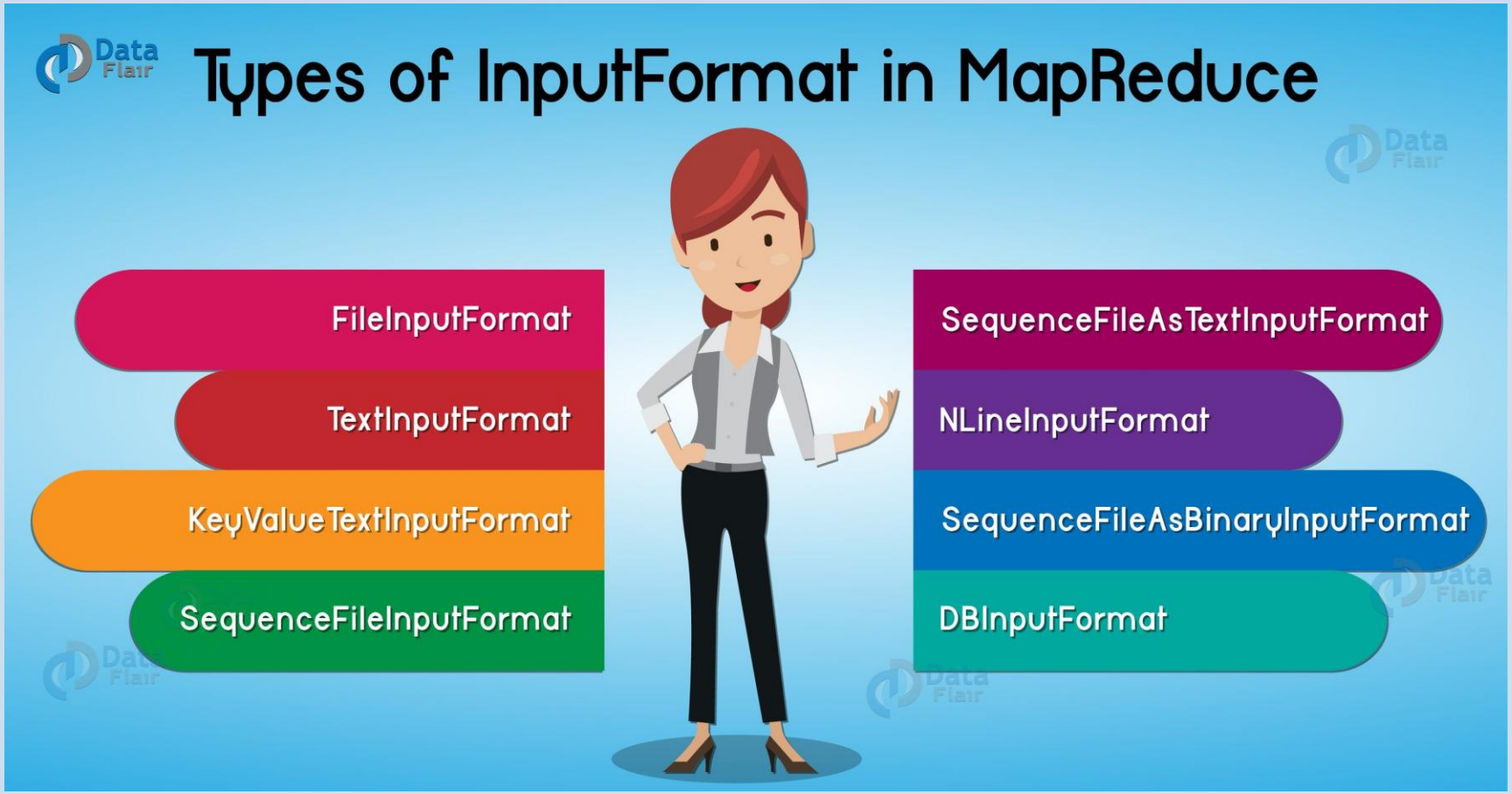



Input
Mapper
Reducer
Output
Node 1
Order1: {Prod1, Prod2, Prod3}
Order2: {Prod7, Prod3, Prod5}
Order6: {Prod1, Prod2, Prod7}
Order10: {Prod4, Prod3, Prod7}
Prod1: [1, 1], Prod2: [1, 1],
Prod3: [1, 1, 1], Prod4: [1],
Prod5: [1], Prod7: [1, 1, 1]
Prod1: [1, 1, 1, 1, 1],
Prod4: [1, 1],
Prod7: [1, 1, 1, 1, 1, 1, 1]
Node 2
Order8: {Prod9, Prod8, Prod6}
Order5: {Prod6, Prod7, Prod8}
Order3: {Prod6, Prod7, Prod1}
Prod8: [1, 1], Prod9: [1],
Prod1: [1], Prod6: [1, 1, 1],
Prod7: [1, 1]
Prod2: [1, 1],
Prod5: [1, 1, 1],
Prod8: [1, 1]
Node 3
Order4: {Prod1, Prod5, Prod6}
Order7: {Prod3, Prod4, Prod7}
Order9: {Prod1, Prod5, Prod7}
Prod1: [1, 1], Prod5: [1, 1],
Prod6: [1], Prod7: [1, 1],
Prod4: [1], Prod3: [1]
Prod3: [1, 1, 1, 1],
Prod6: [1, 1, 1, 1],
Prod9: [1]
Prod1: 5
Prod4: 2
Prod7: 7
Prod2: 2
Prod5: 3
Prod8: 2
Prod3: 4
Prod6: 4
Prod9: 1


Input
Mapper
Combiner
Reducer
Output
Node 1
Order1: {Prod1, Prod2, Prod3}
Order2: {Prod7, Prod3, Prod5}
Order6: {Prod1, Prod2, Prod7}
Order10: {Prod4, Prod3, Prod7}
Prod1: [1, 1],
Prod2: [1, 1],
Prod3: [1, 1, 1],
Prod4: [1],
Prod5: [1],
Prod7: [1, 1, 1]
Prod1: [2],
Prod2: [2],
Prod3: [3],
Prod4: [1],
Prod5: [1],
Prod7: [3]
Prod1: [2, 1, 2],
Prod4: [1, 1],
Prod7: [3, 2, 2]
Node 2
Order8: {Prod9, Prod8, Prod6}
Order5: {Prod6, Prod7, Prod8}
Order3: {Prod6, Prod7, Prod1}
Prod8: [1, 1],
Prod9: [1],
Prod1: [1],
Prod6: [1, 1, 1],
Prod7: [1, 1]
Prod8: [2],
Prod9: [1],
Prod1: [1],
Prod6: [3],
Prod7: [2]
Prod2: [2],
Prod5: [1, 2],
Prod8: [2]
Node 3
Order4: {Prod1, Prod5, Prod6}
Order7: {Prod3, Prod4, Prod7}
Order9: {Prod1, Prod5, Prod7}
Prod1: [1, 1],
Prod5: [1, 1],
Prod6: [1],
Prod7: [1, 1],
Prod4: [1],
Prod3: [1]
Prod1: [2],
Prod5: [2],
Prod6: [1],
Prod7: [2],
Prod4: [1],
Prod3: [1]
Prod3: [3, 1],
Prod6: [3, 1],
Prod9: [1]
Prod1: 5
Prod4: 2
Prod7: 7
Prod2: 2
Prod5: 3
Prod8: 2
Prod3: 4
Prod6: 4
Prod9: 1

MapReduce Job Execution




Data Flair
Types of InputFormat in MapReduce
FileInputFormat
TextInputFormat
KeyValueTextInputFormat
SequenceFileInputFormat
SequenceFileAsTextInputFormat
NLineInputFormat
SequenceFileAsBinaryInputFormat
DBInputFormat

Thank You